Predicting Hotel Booking Cancellations — a CRISP-DM Project

Author: Meron Tesfandrias · **Course:** CMPE 255 · **Repo:** MeronDT/CMPE-255-Data-Mining-process · **App:** cmpe-255-data-mining-process-hotel-cancellations.streamlit.app

Data: *Hotel booking demand datasets* — N. António, A. de Almeida & L. Nunes, *Data in Brief* 22 (2019) 41–49, doi:10.1016/j.dib.2018.11.126. 119,390 bookings for a resort hotel (H1) and a city hotel (H2) in Portugal, arrivals July 2015 – August 2017. Mirrored on Kaggle as `jessemostipak/hotel-booking-demand` and in the TidyTuesday repo (used here — no login needed).

This notebook follows the **CRISP-DM 1.0** reference model end to end: each phase is broken into its **generic tasks**, and each task produces the **outputs** CRISP-DM defines for it.

#	Phase	Generic tasks
1	Business Understanding	Determine business objectives · Assess situation · Determine data mining goals · Produce project plan
2	Data Understanding	Collect initial data · Describe data · Explore data · Verify data quality
3	Data Preparation	Select · Clean · Construct · Integrate · Format data
4	Modeling	Select modeling technique · Generate test design · Build model · Assess model
5	Evaluation	Evaluate results · Review process · Determine next steps
6	Deployment	Plan deployment · Plan monitoring & maintenance · Produce final report · Review project

Headline result. A calibrated random-forest model that estimates each booking's cancellation probability from the 24 fields known when it is booked.

- **What it is used for.** For each arrival date, the probabilities are added up into an **expected number of cancellations**, which tells the hotel how many extra rooms to sell.
- **The business case.** On the most recent 20 % of arrivals, which were never used for training or tuning, this per-night forecast misses the true number of cancellations by **4.6 rooms** on average. The hotel's current rule of thumb misses by 9.4. Under the assumed costs it is worth **€3,400 more per 1,000 bookings** than that rule (95 % CI €2,400–€4,300).
- **What it is not good at.** As a *per-booking* classifier it misses two of its three data mining targets: ROC-AUC is **0.827** (target 0.85) and recall at the cost-optimal threshold is 0.47 (target 0.50). Precision is 0.83 (target 0.75). So it is approved for the overbooking forecast, not for acting on individual bookings.

- **What changed it.** The most predictive field, guest nationality, was **excluded**. A test in the data showed it was probably filled in by default for guests who never arrived, which leaks the outcome.

It is deployed as a Streamlit app that reads raw booking-system fields. (The test period was scored once in the first iteration and was visible during the later review rounds; see 4.2.)

How to run: *Runtime → Run all* (≈ 7–10 min on free Colab). All figures, reports, the model and the app are written to `artifacts/`.

⚙️ Setup & configuration

In [1]:
```
import json, platform, warnings
from datetime import datetime
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
from scipy.stats import loguniform

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 60)
plt.rcParams["figure.figsize"] = (8, 4.5)
print(f"python {platform.python_version()} | numpy {np.__version__} | pandas {pd
```

```
python 3.13.18 | numpy 2.5.3 | pandas 3.0.5 | scikit-learn 1.9.1
```

In [2]:
```
# ============================ CONFIGURATION ============================
BASE_URL = "https://raw.githubusercontent.com/rfordatascience/tidytuesday/main/d
SOURCES  = {"Resort Hotel": BASE_URL + "H1.csv", "City Hotel": BASE_URL + "H2.cs
COMBINED_REFERENCE = BASE_URL + "hotels.csv"   # published combined file, used o

TARGET       = "is_canceled"   # 1 = booking cancelled
RANDOM_STATE = 42
TEST_SIZE    = 0.20            # most recent 20 % of arrivals held out (time-bas
CV_FOLDS     = 5
N_TUNING_ITER = 8              # random-search iterations per model family (the

# Phase 1.3 data mining success criteria. ROC-AUC and precision were fixed befor
# added in revision 2 (critiques/round_2.md) because precision alone can be met
SUCCESS_CRITERIA = {"roc_auc": 0.85, "precision": 0.75, "recall": 0.50}

# Phase 1.2 cost/benefit assumptions. The model's ONE action is internal: rooms
# allowance (guests are never contacted, so nationality never drives a guest-fac
# the same event - the hotel filling up that night - so both are weighted by P_F
P_FULL       = 0.5      # share of nights on which demand would fill the hotel (
RESALE_VALUE = 100.0    # a freed room re-sold on a full night (about one night'
WALK_COST    = 200.0    # walking a guest on a full night: room elsewhere, trans
VALUE_TP = RESALE_VALUE * P_FULL   # expected value of a correct flag  = EUR 50
COST_FP  = WALK_COST * P_FULL      # expected cost of a wrong flag     = EUR 100
COST_FN  = 0.0                     # missed cancellation = today's status quo
# ======================================================================
```

```
ART = Path("artifacts"); FIG, REP, MOD, APP = ART / "figures", ART / "reports",
for p in (FIG, REP, MOD, APP):
    p.mkdir(parents=True, exist_ok=True)

def savefig(name):
    plt.tight_layout(); plt.savefig(FIG / f"{name}.png", dpi=120, bbox_inches="t
```

Phase 1 — Business Understanding

Goal: understand the project objectives from a business perspective, convert them into a data mining problem definition, and draw up a preliminary plan.

1.1 Determine business objectives

Output: Background

The stakeholder is the revenue-management team of two hotels in Portugal — a resort hotel in the Algarve (H1) and a city hotel in Lisbon (H2) — whose booking records were published by António, de Almeida & Nunes (2019). Across July 2015 – August 2017, **37 % of all bookings were cancelled** (42 % in the city hotel). Cancellations are a structural problem in hospitality: free-cancellation policies and online travel agencies make it cheap for guests to book speculatively. A room released a few days before arrival is often impossible to re-sell. The usual defence is to overbook each night by a fixed rule of thumb (the historical cancellation rate), and that risks "walking" guests (relocating them to another hotel) when more people turn up than expected. Today, neither hotel estimates cancellation risk per booking. At roughly 55,000 bookings a year and €346 of room revenue per arrived booking (2.3), cancellations touch about €7 M of booked revenue a year across the two hotels.

Output: Business objectives

- **Primary objective:** reduce the revenue lost to cancellations by **setting each night's overbooking level from the bookings actually on the books**, instead of from a fixed historical rate. The model's output is used internally only; guests are not contacted on the basis of a score.
- **Related business questions:** Which booking characteristics drive cancellation? Do the city and resort hotels behave differently? Which channels and customer types are riskiest?
- **Expected benefits:** more re-sold rooms and fewer walked guests than the current rule of thumb.

Output: Business success criteria

Criterion	Measure	Target	Judged by
Revenue recovered	net value per 1,000 bookings of the overbooking decisions, simulated per night with the cost model in 1.2	above the **hotel's current rule of thumb**, with the 95 % confidence interval of the difference above zero	Revenue manager
Forecast accuracy	average error of the nightly overbooking number (rooms per night)	smaller than the current rule's error	Revenue manager
Actionability	share of flagged high-risk bookings that really cancel	≥ 75 %	Front-office manager
Usability	staff can score the day's booking file without technical help	3 reservations staff score a day's file unaided in under 5 minutes during the pilot	Reservations team

Revisions 2 and 3 replaced the original benchmarks ("flag everything", then the Non Refund rule) with the incumbent rule of thumb, which is what the hotel actually does today.

1.2 Assess situation

Output: Inventory of resources

Resource	Available
Personnel	the analyst (author); an AI assistant (Claude) for code review and expert critique; the course instructor. **No hotel stakeholder or finance contact** (see risks)
Data	2 CSV files (H1 resort 40,060 rows; H2 city 79,330 rows), 31 attributes, open data
Computing	Google Colab (free tier), GitHub
Software	Python, pandas, scikit-learn, matplotlib, Streamlit; optionally IBM SPSS Modeler

Output: Requirements, assumptions and constraints

- **Requirements:** predictions may use **only information known when the booking is made**; results must be explainable to hotel staff; the model must be deployable as a simple web app that accepts the hotel's own booking fields; the work must be reproducible end to end.
- **Legal and ethical requirement:** nationality is personal data under GDPR, and treating guests differently by nationality would be discriminatory. The output may therefore drive only the internal overbooking level, never a guest-facing action such as a deposit request. (After 2.3, nationality is not a model input at all.)
- **Assumptions:** 2015–2017 booking behaviour is representative of future bookings; the published data is accurate (hotel and customer identifiers were removed by the authors).
- **Constraints:** no booking ID, no explicit booking date (it can be derived as arrival date − lead time, see 5.1), no price history, no customer history beyond two counts; public data only; free-tier compute.

Output: Risks and contingencies

Risk	Likelihood	Impact	Contingency
Target leakage from fields filled in after booking	High	High	audit each attribute for "known at booking time?" (2.4, 3.1), test suspicious fields in the data (2.3) and their effect (5.2)
No access to the business: costs, demand and recording practices are all assumed	High	High	cost sensitivity in 5.1; validate the assumptions at a pilot kick-off before go-live
2015–17 behaviour no longer representative (pre-pandemic, before today's OTA and free-cancellation norms)	High	High	treat this model as a proof of concept; retrain on the hotel's recent booking-system data before go-live (6.2)
Behaviour shifts over time (concept drift)	Medium	High	time-based test split and time-series validation; drift monitoring (6.2)
Model misses the success criteria	Medium	Medium	feature construction, tuning; revisit goals (5.3)
Data-quality quirks (e.g. odd deposit codes)	Medium	Medium	investigate and document (2.3, 2.4)

Output: Terminology

Term	Meaning
ADR	Average Daily Rate — revenue per occupied room-night (EUR)
Lead time	days between booking and arrival
No-show	guest neither cancelled nor arrived (counted as a cancellation here)
Overbooking / "walking" a guest	selling more rooms than exist; relocating a guest when everyone shows up
Expected cancellations	the sum of the cancellation probabilities of the bookings arriving on a date
ROC-AUC	probability that the model ranks a random cancellation above a random non-cancellation
Precision / recall	share of flagged bookings that really cancel / share of cancellations that are flagged
Calibration	whether a predicted 30 % really means that 30 % of such bookings cancel

Output: Costs and benefits

An arrived booking brings about **€346** of room revenue (mean of ADR × nights over arrived bookings, 2.3; median €259). Both outcomes of an overbooking decision depend

on the same event: the hotel filling up that night. Both are therefore weighted by the same probability `P_FULL`:

Outcome	Business meaning	Assumed value	Reasoning
A room that was oversold is freed by a cancellation	it is re-sold, if demand fills the hotel	+€100 × 50 % = **+€50**	about one night's ADR (€100), on a full night
A room that was oversold is NOT freed (the guest arrives)	a guest is walked, if the hotel is full	−€200 × 50 % = **−€100**	a comparable room elsewhere, transport and compensation (about 2 nights' ADR), on a full night
Cancellation not anticipated	same as today	€0	the loss happens with or without the model
Booking arrives, not overbooked	normal stay	€0	—

A wrong call therefore costs twice what a right call earns. These are assumptions, not hotel figures, so 5.1 re-derives the threshold and the value for cost ratios from 0.5 to 4. *Revision 3 applied `P_FULL` to both sides; revision 2 had applied it to the walk cost only.*

1.3 Determine data mining goals

Output: Data mining goals

Build a **binary classifier** that estimates, **at booking time**, the probability that a booking will be cancelled (`is_canceled = 1`, including no-shows). One row = one booking. The probabilities must be **calibrated**, because they are added up into the nightly expected cancellations. Calibration is judged descriptively (reliability curve, Brier score) and through the forecast-accuracy business criterion. Secondary goal: identify the main drivers of cancellation.

Output: Data mining success criteria

Metric	Threshold	Why
ROC-AUC on the most recent 20 % of arrivals	≥ 0.85	the model must rank risky bookings well
Precision at the chosen decision threshold	≥ 0.75	3 of 4 flagged bookings must truly cancel, or staff will ignore the flags
Recall at the chosen decision threshold	≥ 0.50	at least half of all cancellations must be caught; without this floor the precision target could be met by flagging almost nothing. Added in revision 2, after the first results were known

1.4 Produce project plan

Output: Project plan

Phase	Duration (planned)	Resources	Outputs	Risks / dependencies
Business Understanding	1 day	analyst, AI critic	sections 1.1–1.4	—
Data Understanding	1 day	Colab	data description, exploration and quality reports	data access
Data Preparation	1–2 days	Colab	modeling dataset + reports	leakage decisions
Modeling	1 day	Colab	candidate and tuned models	compute time on free tier
Evaluation	1 day	analyst, AI critic	approved model, business case	criteria not met → iterate
Deployment	1–2 days	Streamlit Cloud, GitHub	app, monitoring plan, report, article, video	hosting

Iterations expected: Data Preparation ↔ Modeling (feature and leakage decisions), and Evaluation → Business Understanding if the criteria are not met. Both happened, over three review rounds:

- the **test design** was changed after the first evaluation (4.2);
- the business case was rebuilt around **one action** (the nightly overbooking level) and the hotel's **current rule** as benchmark;
- a test in the data led back from Evaluation to Data Preparation: **nationality was excluded** as probable leakage (2.3, 3.1).

Output: Initial assessment of tools and techniques

Python and scikit-learn in Colab make the whole project reproducible from one notebook. Candidate techniques: **logistic regression** as an interpretable benchmark; **random forest** and **gradient boosting** for non-linear effects and interactions (e.g. lead time × deposit type × market segment). Streamlit gives a free, code-based deployment. IBM SPSS Modeler would support the same CRISP-DM flow visually but adds licensing; it was not needed.

🔍 Critic review — Business Understanding

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on CRISP-DM, a sought-after keynote speaker, and the author of several award-winning books on the CRISP-DM methodology. Review the **Business Understanding** phase of my hotel-cancellation project below. Check it against every generic task and output CRISP-DM 1.0 defines for this phase. Focus especially on: whether the business objective, business success criteria and data mining success criteria are measurable and consistent with each other, and whether the cost/benefit assumptions are realistic and justified. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Cost figures were unjustified and the false-positive cost (€40) looked too low next to the cost of walking a guest; precision target needed a rationale.	Derived costs from the average booking (€358): +€100 / −€100; justified the 75 % precision target as staff trust.
2	One flag served three different actions priced alike; no recall floor; "beat flag-all" a straw man; no legal requirement on nationality; no cost sensitivity. (C+ overall)	One internal action; walk cost × P(full); recall ≥ 0.50; Non Refund rule as benchmark; GDPR requirement; cost-ratio grid.
3	P(full) applied to the walk cost only; benchmark should be the hotel's current overbooking rule; no stakeholder-access risk. (B− overall)	Both outcomes × P(full) → 2 : 1 ratio; business criterion = beat the current rule per night (CI > 0); new risk row.

Phase 2 — Data Understanding

Goal: collect the data, describe and explore it, and verify its quality.

2.1 Collect initial data

In [3]:
```
originals = {hotel: pd.read_csv(url) for hotel, url in SOURCES.items()}
collection = pd.DataFrame([{"hotel": h, "source": SOURCES[h].split("/")[-1], "ro
                            for h, d in originals.items()])
collection.to_csv(REP / "initial_data_collection.csv", index=False)
display(collection)
originals["Resort Hotel"].head(3)
```

	hotel	source	rows	columns
0	Resort Hotel	H1.csv	40060	31
1	City Hotel	H2.csv	79330	31

Out[3]:

	IsCanceled	LeadTime	ArrivalDateYear	ArrivalDateMonth	ArrivalDateWeekNumber	A
0	0	342	2015	July	27	
1	0	737	2015	July	27	
2	0	7	2015	July	27	

Output: Initial data collection report

Item	Detail
Source	António, de Almeida & Nunes (2019), *Data in Brief*; files H1.csv (resort) and H2.csv (city) via the TidyTuesday mirror
Retrieval	direct HTTPS download with pandas (no credentials)
Size	40,060 + 79,330 = 119,390 bookings, 31 attributes each
Problems encountered	two separate files with **no hotel identifier**; inconsistent formatting between and within them (see 2.2)
Data wanted but unavailable	booking ID; an explicit booking date (it can be derived as arrival date − lead time); price paid vs. list price; customer history across stays; local events and weather

2.2 Describe data

In [4]:
```
# How consistent are the two files?
a, b = originals["Resort Hotel"], originals["City Hotel"]
padded = lambda s: s.astype(str).str.match(r"^\s|.*\s$").any()
fmt = pd.DataFrame({"resort_dtype": a.dtypes.astype(str), "city_dtype": b.dtypes
fmt["padded_text"] = [padded(a[c]) or padded(b[c]) for c in a.columns]
fmt["literal_NULL"] = [int((a[c].astype(str).str.strip() == "NULL").sum() + (b[c
inconsistent = fmt[(fmt.resort_dtype != fmt.city_dtype) | fmt.padded_text | (fmt
print("Columns with format problems:"); display(inconsistent)
```

```
Columns with format problems:
```

	resort_dtype	city_dtype	padded_text	literal_NULL
Children	int64	float64	False	0
Meal	str	str	True	0
ReservedRoomType	str	str	True	0
AssignedRoomType	str	str	True	0
DepositType	str	str	True	0
Agent	str	str	True	16340
Company	str	str	True	112593

In [5]:
```
import re

def harmonize(d, hotel):
    """Standardise one source file: snake_case names, trimmed text, 'NULL' -> mi
```

```
    d = d.copy()
    d.columns = [re.sub(r"(?<!^)(?=[A-Z][a-z])", "_", c).lower() for c in d.colu
    for c in d.select_dtypes(exclude="number").columns:
        d[c] = d[c].astype(str).str.strip().replace({"NULL": np.nan, "nan": np.n
    for c in ("agent", "company", "children"):
        d[c] = pd.to_numeric(d[c], errors="coerce")
    d.insert(0, "hotel", hotel)
    return d

# Stack both hotels for exploration (formally validated in 3.4 Integrate data)
raw = pd.concat([harmonize(d, h) for h, d in originals.items()], ignore_index=Tr

MEANING = {
    "hotel": "Resort Hotel (H1) or City Hotel (H2)", "is_canceled": "TARGET: 1 i
    "lead_time": "days between booking and arrival", "arrival_date_year": "arriv
    "arrival_date_month": "arrival month", "arrival_date_week_number": "arrival
    "arrival_date_day_of_month": "arrival day", "stays_in_weekend_nights": "Sat/
    "stays_in_week_nights": "Mon–Fri nights booked", "adults": "number of adults
    "babies": "number of babies", "meal": "meal package (BB, HB, FB, SC/Undefine
    "country": "country of origin (ISO 3166-1 alpha-3)", "market_segment": "mark
    "distribution_channel": "booking distribution channel", "is_repeated_guest":
    "previous_cancellations": "prior bookings cancelled by this customer", "prev
    "reserved_room_type": "room type reserved (code)", "assigned_room_type": "ro
    "booking_changes": "changes made to the booking until check-in/cancellation"
    "agent": "ID of travel agency (missing = none)", "company": "ID of company p
    "days_in_waiting_list": "days on waiting list before confirmation", "custome
    "adr": "average daily rate (EUR)", "required_car_parking_spaces": "parking s
    "total_of_special_requests": "special requests made", "reservation_status":
    "reservation_status_date": "date the last status was set",
}
dd = pd.DataFrame({"meaning": pd.Series(MEANING), "dtype": raw.dtypes.astype(str
                   "missing_%": (raw.isna().mean() * 100).round(2), "n_unique":
                   "example": raw.apply(lambda s: s.dropna().iloc[0] if s.notna(
dd.to_csv(REP / "data_description.csv")
print(f"{raw.shape[0]:,} bookings x {raw.shape[1]} attributes"); display(dd)
```

```
119,390 bookings x 32 attributes
```

	meaning	dtype	missing_%	n_unique	example
hotel	Resort Hotel (H1) or City Hotel (H2)	str	0.00	2	Resort Hotel
is_canceled	TARGET: 1 if cancelled (incl. no-show)	int64	0.00	2	0
lead_time	days between booking and arrival	int64	0.00	479	342
arrival_date_year	arrival year	int64	0.00	3	2015
arrival_date_month	arrival month	str	0.00	12	July
arrival_date_week_number	arrival ISO week	int64	0.00	53	27
arrival_date_day_of_month	arrival day	int64	0.00	31	1
stays_in_weekend_nights	Sat/Sun nights booked	int64	0.00	17	0
stays_in_week_nights	Mon–Fri nights booked	int64	0.00	35	0
adults	number of adults	int64	0.00	14	2
children	number of children	float64	0.00	5	0.0
babies	number of babies	int64	0.00	5	0
meal	meal package (BB, HB, FB, SC/Undefined = none)	str	0.00	5	BB
country	country of origin (ISO 3166-1 alpha-3)	str	0.41	177	PRT
market_segment	market segment (TA = travel agent, TO = tour o...	str	0.00	8	Direct
distribution_channel	booking distribution channel	str	0.00	5	Direct
is_repeated_guest	1 if guest stayed before	int64	0.00	2	0

	meaning	dtype	missing_%	n_unique	example
previous_cancellations	prior bookings cancelled by this customer	int64	0.00	15	0
previous_bookings_not_canceled	prior bookings not cancelled	int64	0.00	73	0
reserved_room_type	room type reserved (code)	str	0.00	10	C
assigned_room_type	room type assigned at check-in (code)	str	0.00	12	C
booking_changes	changes made to the booking until check-in/can...	int64	0.00	21	3
deposit_type	No Deposit / Non Refund / Refundable	str	0.00	3	No Deposit
agent	ID of travel agency (missing = none)	float64	13.69	333	304.0
company	ID of company paying (missing = none)	float64	94.31	352	110.0
days_in_waiting_list	days on waiting list before confirmation	int64	0.00	128	0
customer_type	Transient, Contract, Group, Transient-Party	str	0.00	4	Transient
adr	average daily rate (EUR)	float64	0.00	8879	0.0
required_car_parking_spaces	parking spaces requested	int64	0.00	5	0
total_of_special_requests	special requests made	int64	0.00	6	0

	meaning	dtype	missing_%	n_unique	example
reservation_status	last status: Canceled, Check-Out, No-Show	str	0.00	3	Check-Out
reservation_status_date	date the last status was set	str	0.00	926	2015-07-01

Output: Data description report

The integrated data has **119,390 bookings × 32 attributes** (31 original + `hotel`): 16–17 integer, 1–2 float and 13 text fields per file. Units: money in EUR (`adr`), durations in days or nights.

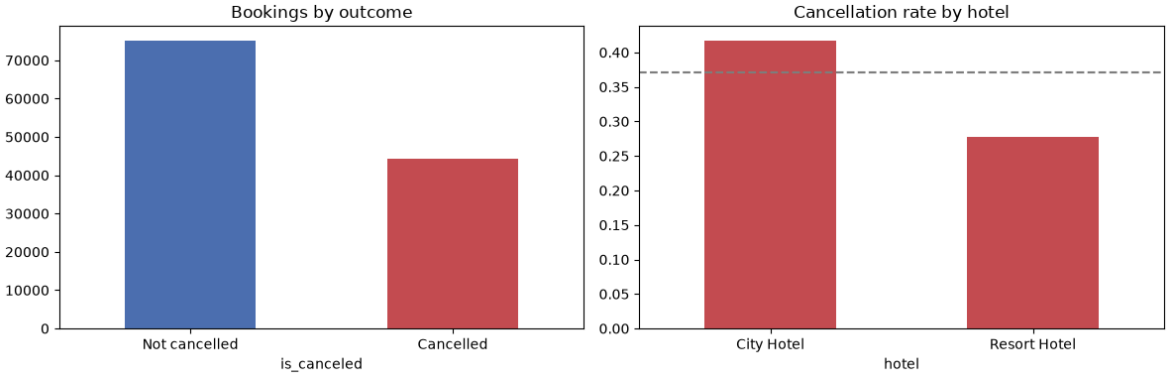
Format problems found and fixed in `harmonize()`: `Children` is an integer in one file and a float in the other; `Meal`, `ReservedRoomType`, `AssignedRoomType` and `DepositType` are padded with spaces; `Agent` and `Company` store missing values as the text `"NULL"` (16,340 and 112,593 times). Column names are CamelCase in the source and are converted to snake_case.

Against the Phase 1 requirement "known at booking time", three groups emerge: clearly known at booking (lead time, dates, stay length, guests, meal, segment, channel, deposit, ADR…); **only known after the outcome** (`reservation_status`, `reservation_status_date`, `assigned_room_type`); and **ambiguous** (`booking_changes`, `required_car_parking_spaces`, `total_of_special_requests`, `country`), which can be updated between booking and arrival. 2.4 and 5.2 deal with these.

2.3 Explore data

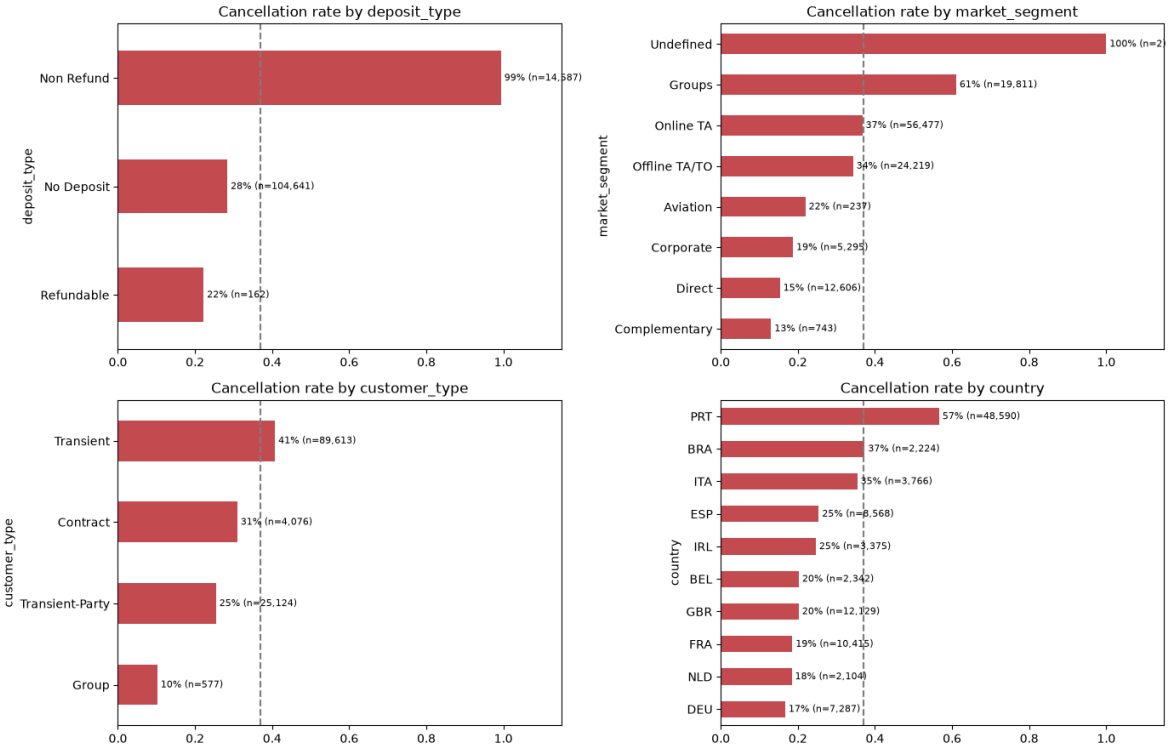
In [6]:
```
rate = raw[TARGET].mean()
arrived = raw[raw[TARGET] == 0]
stay_value = arrived["adr"] * (arrived["stays_in_week_nights"] + arrived["stays_
print(f"Overall cancellation rate: {rate:.1%}")
print(f"Revenue per arrived booking (ADR x nights): mean EUR {stay_value.mean():
      f"mean ADR EUR {arrived['adr'].mean():.1f}")
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
raw[TARGET].map({0: "Not cancelled", 1: "Cancelled"}).value_counts().plot.bar(ax
raw.groupby("hotel")[TARGET].mean().plot.bar(ax=axes[1], color="#C44E52", rot=0,
axes[1].axhline(rate, ls="--", c="gray"); savefig("eda_target")
```

```
Overall cancellation rate: 37.0%
Revenue per arrived booking (ADR x nights): mean EUR 346, median EUR 259; mean AD
R EUR 100.0
```



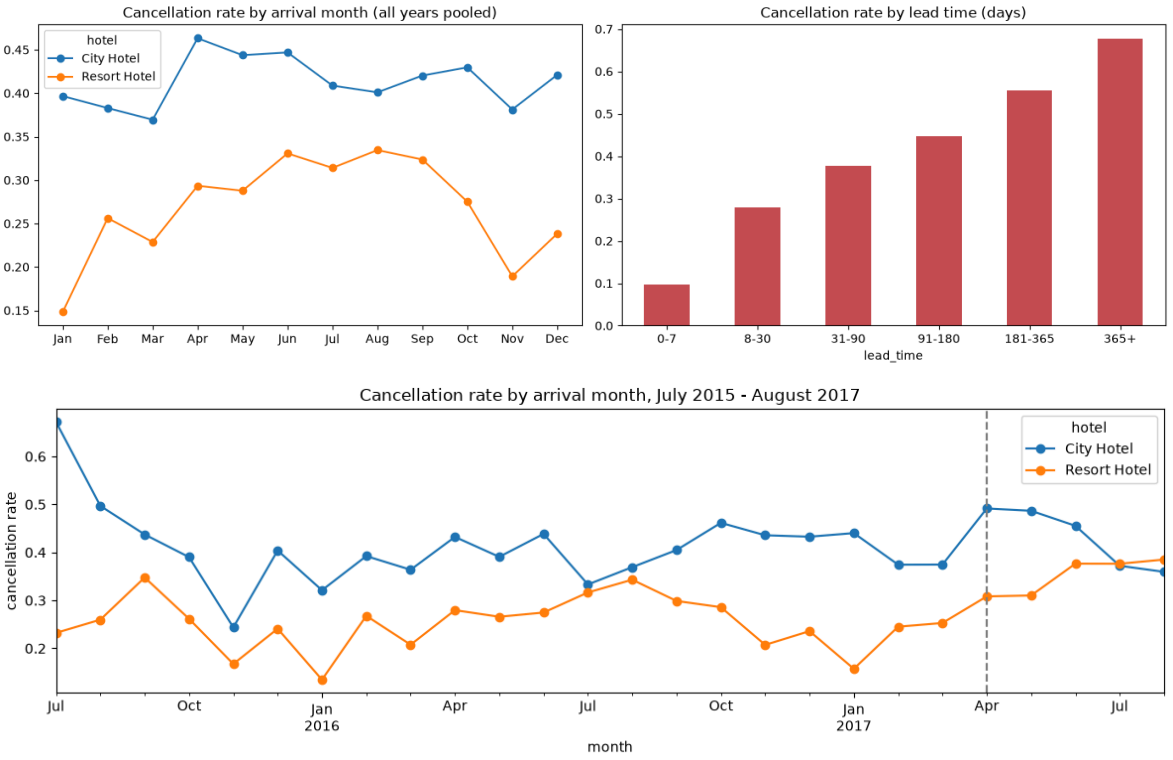
In [7]:
```
def rate_plot(col, order=None, top=None, ax=None):
    g = raw.groupby(col)[TARGET].agg(["mean", "size"])
    if top: g = g.sort_values("size", ascending=False).head(top)
    g = g.reindex(order) if order else g.sort_values("mean")
    g["mean"].plot.barh(ax=ax, color="#C44E52", title=f"Cancellation rate by {co
    for i, (m, n) in enumerate(zip(g["mean"], g["size"])):
        ax.text(m, i, f" {m:.0%} (n={n:,})", va="center", fontsize=8)
    ax.axvline(rate, ls="--", c="gray"); ax.set_xlim(0, 1.15)

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
rate_plot("deposit_type", ax=axes[0, 0]); rate_plot("market_segment", ax=axes[0,
rate_plot("customer_type", ax=axes[1, 0]); rate_plot("country", top=10, ax=axes[
savefig("eda_rates_categorical")
```



In [8]:
```
months = ["January", "February", "March", "April", "May", "June", "July", "Augus
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
raw.pivot_table(index="arrival_date_month", columns="hotel", values=TARGET).rein
   .plot(ax=axes[0], marker="o", title="Cancellation rate by arrival month (all
axes[0].set_xticks(range(12), [m[:3] for m in months])
bins = [-1, 7, 30, 90, 180, 365, 1000]
raw.groupby(pd.cut(raw["lead_time"], bins, labels=["0-7", "8-30", "31-90", "91-1
   .plot.bar(ax=axes[1], color="#C44E52", rot=0, title="Cancellation rate by lea
savefig("eda_month_leadtime")
```

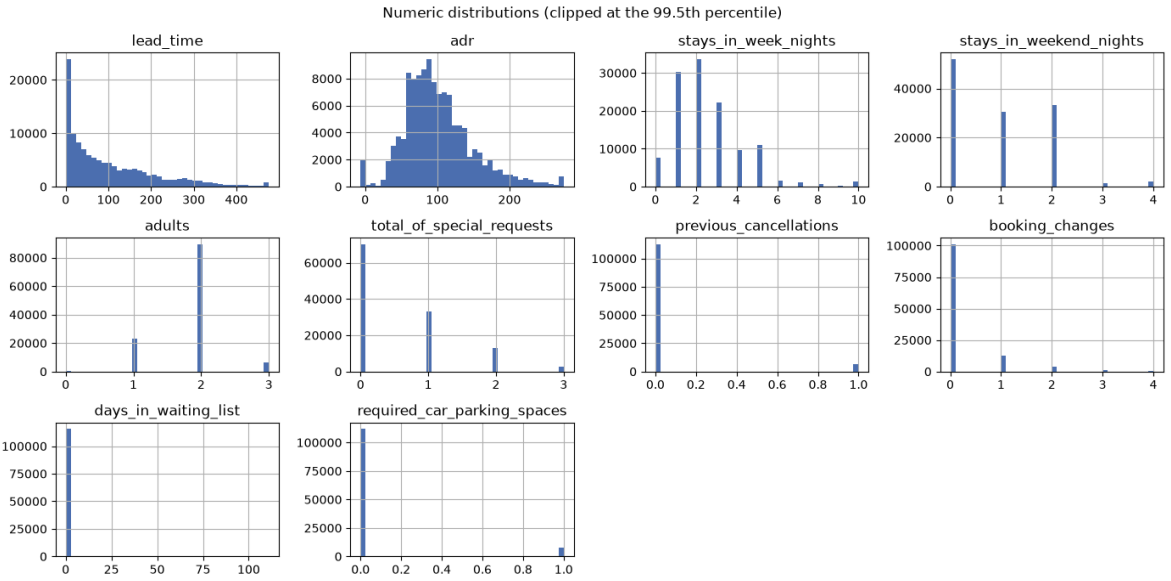
```
# Drift check: the same rate on the real calendar timeline (not pooled by month)
arrival_month = pd.to_datetime(raw["arrival_date_year"].astype(str) + "-" + raw[
timeline = raw.assign(month=arrival_month).pivot_table(index="month", columns="h
fig, ax = plt.subplots(figsize=(12, 4))
timeline.plot(ax=ax, marker="o", title="Cancellation rate by arrival month, July
arrival_day = pd.to_datetime(raw["arrival_date_year"].astype(str) + "-" + raw["a
ax.axvline(arrival_day.quantile(1 - TEST_SIZE), c="gray", ls="--"); ax.set_ylabe
savefig("eda_timeline")
# Like-for-like drift: the same months (April-August) in 2016 and 2017, booking-
same = raw[raw["arrival_date_month"].isin(months[3:8]) & raw["arrival_date_year"
display(same.pivot_table(index="arrival_date_year", columns="hotel", values=TARG
```

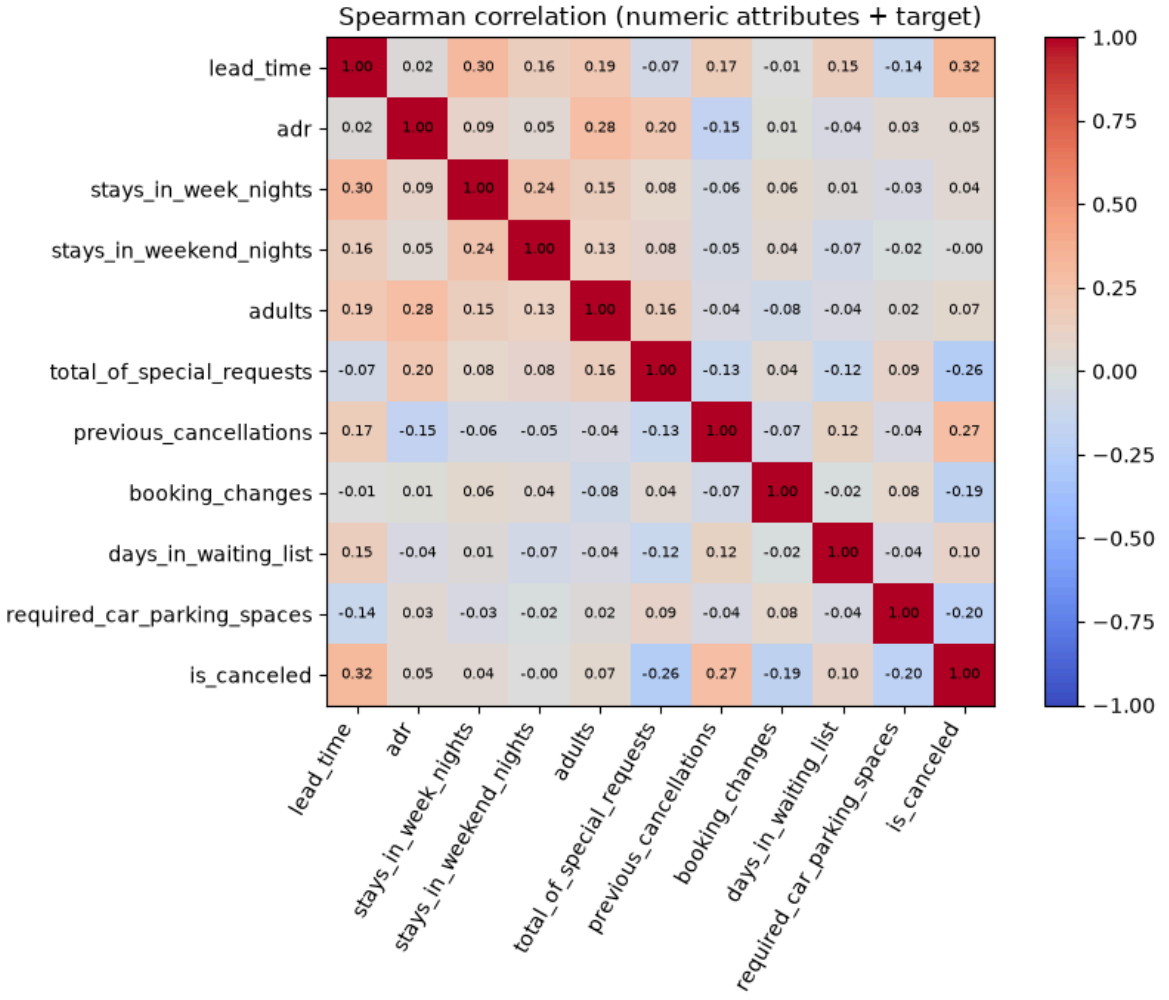


hotel	City Hotel	Resort Hotel
April-August of		
2016	0.395	0.295
2017	0.439	0.351

In [9]:
```
num_cols = ["lead_time", "adr", "stays_in_week_nights", "stays_in_weekend_nights
            "previous_cancellations", "booking_changes", "days_in_waiting_list",
clipped = raw[num_cols].clip(upper=raw[num_cols].quantile(0.995), axis=1)      #
clipped.hist(bins=40, figsize=(14, 7), layout=(3, 4), color="#4C72B0"); plt.supt
savefig("eda_numeric_hist")

corr = raw[num_cols + [TARGET]].corr(method="spearman")
fig, ax = plt.subplots(figsize=(9, 7))
im = ax.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
ax.set_xticks(range(len(corr)), corr.columns, rotation=60, ha="right"); ax.set_y
for i in range(len(corr)):
    for j in range(len(corr)):
        ax.text(j, i, f"{corr.iat[i, j]:.2f}", ha="center", va="center", fontsiz
fig.colorbar(im); ax.set_title("Spearman correlation (numeric attributes + targe
```





In [10]:
```
# Investigate the 'Non Refund' anomaly and the exact-duplicate rows instead of j
dup = raw.duplicated(keep=False)
top_agents = raw["agent"].value_counts().index[:5]
groups = {"Non Refund": raw["deposit_type"] == "Non Refund", "exact-duplicate ro
          "duplicates, not Non Refund": dup & (raw["deposit_type"] != "Non Refun
profile = lambda d: [len(d), d[TARGET].mean(), (d["country"] == "PRT").mean(),
                     d["market_segment"].isin(["Groups", "Offline TA/TO"]).mean(
                     d.duplicated(keep=False).mean(), d["agent"].isin(top_agents
                     d["total_of_special_requests"].mean(), d["lead_time"].media
nr_table = pd.DataFrame({name: profile(raw[m]) for name, m in groups.items()},
                        index=["bookings", "cancellation rate", "share from Port
```

```
                                "share Non Refund", "share that are exact duplica
                                "mean special requests", "median lead time (days)
nr_table.to_csv(REP / "non_refund_investigation.csv"); display(nr_table)
arrived_prt, cancelled_prt = (raw.loc[raw[TARGET] == 0, "country"] == "PRT").mea
print(f"Share of Portuguese guests: {cancelled_prt:.0%} of cancelled bookings vs
# If nationality were only filled in at check-in, bookings that never checked in
# PRT share by final status inside each market segment, excluding the Non Refund
# sharpest contrast: neither no-shows nor cancellations ever reached the front d
nn = raw[raw["deposit_type"] != "Non Refund"]
prt_test = nn.assign(PRT=nn["country"] == "PRT").pivot_table(index="market_segme
prt_test["bookings"] = nn["market_segment"].value_counts()
prt_test = prt_test[prt_test["bookings"] >= 1000].round(3); prt_test.to_csv(REP
nr_dates = raw[groups["Non Refund"]].groupby(["arrival_date_year", "arrival_date
print(f"Non Refund bookings fall on {len(nr_dates):,} arrival dates; the 20 busi
      f"{nr_dates.nlargest(20).sum() / nr_dates.sum():.0%} of them")
```

	Non Refund	exact-duplicate rows	duplicates, not Non Refund	all bookings
bookings	14587.000	40165.000	25849.000	119390.000
cancellation rate	0.994	0.584	0.355	0.370
share from Portugal	0.972	0.606	0.402	0.407
share Groups or Offline TA/TO	0.972	0.720	0.578	0.369
share Non Refund	1.000	0.356	0.000	0.122
share that are exact duplicates	0.981	1.000	1.000	0.336
share via the 5 largest agents	0.281	0.361	0.403	0.505
mean special requests	0.002	0.287	0.445	0.571
median lead time (days)	183.000	133.000	101.000	69.000

```
Share of Portuguese guests: 62% of cancelled bookings vs 28% of arrived ones
```

reservation_status	Canceled	Check-Out	No-Show	bookings
market_segment				
Corporate	0.952	0.731	0.934	4961
Direct	0.615	0.435	0.665	12587
Groups	0.968	0.288	0.973	10639
Offline TA/TO	0.948	0.243	0.986	19213
Online TA	0.257	0.183	0.371	56421

```
Non Refund bookings fall on 403 arrival dates; the 20 busiest dates hold 20% of t
hem
```

Output: Data exploration report

#	Finding	Hypothesis / implication	Tested?
1	The city hotel cancels far more than the resort (41.7 % vs 27.8 %)	include `hotel`; evaluate performance per hotel (5.1)	yes, 5.1
2	Cancellation rises steadily with lead time: 9.6 % for bookings made ≤ 7 days ahead, 56 % at 6–12 months, 68 % beyond a year	`lead_time` will be a key driver	yes: alone it ranks cancellations with AUC 0.70 (2.4)
3	**Non Refund deposits cancel 99.4 % of the time**, the opposite of what a non-refundable deposit should do. The 14,587 bookings are 97 % from Portugal and 97 % Groups or offline tour-operator bookings. They have almost no special requests, a median lead time of 183 days, and **98 % of them are exact duplicate rows**. They are spread over 403 arrival dates.	Most likely **block allocations** by tour operators (many identical rooms on one contract), recorded as "non refund" and later released. Keep the attribute (it is known at booking), report it as a data-quality question for the hotel, and show results without these bookings (5.1)	partly: supported by the duplicate profile; only the hotel can confirm
4	Guests from Portugal: 62 % of cancelled bookings vs 28 % of arrived ones. **Test for a defaulted value** (printed table, Non Refund excluded): within every market segment, bookings that never reached the front desk (cancellations *and* no-shows) are Portuguese far more often than checked-out ones. Groups: 97 % / 97 % vs 29 %. Offline TA/TO: 95 % / 99 % vs 24 %. Corporate: 95 % / 93 % vs 73 %.	A real nationality effect would not make no-shows look like cancellations and unlike arrivals *within the same segment*. The pattern fits nationality being **filled in by default (Portugal) unless confirmed at check-in**, so `country` partly encodes the outcome	**yes: supported → `country` is excluded (3.1)**; the effect of adding it back is measured in 5.2
5	Groups (61 %) and online travel agents (37 %) cancel most; direct (15 %) and corporate (19 %) bookings least; more special requests go with fewer cancellations (rank-correlation matrix)	keep `market_segment`, `distribution_channel`, `total_of_special_requests`	yes, 5.1 drivers
6	Seasonality differs by hotel: the resort's cancellation rate ranges from 15 % (January) to 33 % (August); the city hotel stays at 37–46 % year round	keep month/week; expect seasonal drift in monitoring	yes, 6.2

#	Finding	Hypothesis / implication	Tested?
7	**Like for like, cancellations rose:** April–August 2017 vs April–August 2016, the city hotel went from 39.5 % to 43.9 % and the resort from 29.5 % to 35.1 %	behaviour drifts upward, so the most recent period will be harder; this motivates the time-based test split	yes: the test period is harder than CV (5.1)
8	**Exact duplicate rows** (34 % of all rows when both copies are counted) are 36 % Non Refund. The duplicates **outside** Non Refund cancel at 35.5 % (like all bookings) and are 58 % Groups / offline tour-operator bookings	duplicates are mostly multi-room group bookings, not data errors, so they are kept, but they must never straddle a validation fold (4.2)	yes, 4.2

2.4 Verify data quality

In [11]:
```
from sklearn.metrics import roc_auc_score
n = len(raw)
quality = pd.DataFrame({"missing": raw.isna().sum(), "missing_%": (raw.isna().me
nights = raw["stays_in_week_nights"] + raw["stays_in_weekend_nights"]
issues = {
    "Exact duplicate rows": int(raw.duplicated().sum()),
    "Bookings with 0 guests (adults+children+babies)": int((raw[["adults", "chil
    "Negative ADR": int((raw["adr"] < 0).sum()),
    "ADR > 1,000 EUR": int((raw["adr"] > 1000).sum()),
    "ADR = 0 outside the Complementary segment": int(((raw["adr"] == 0) & (raw["
    "Bookings with 0 nights": int((nights == 0).sum()),
    "Stays longer than 30 nights": int((nights > 30).sum()),
    "More than 10 adults on one booking": int((raw["adults"] > 10).sum()),
    "9 or more children or babies on one booking": int(((raw["children"] >= 9) |
    "Meal = 'Undefined' (same as 'SC' per data paper)": int((raw["meal"] == "Und
}
print(pd.Series(issues).to_string()); display(quality[quality["missing"] > 0])
pd.Series(issues, name="count").to_csv(REP / "data_quality_issues.csv")
quality.to_csv(REP / "data_quality_missing.csv")

def leakage_suspects(df, target, max_levels=30, min_share=0.01):
    """Flag attributes where a sizeable group has a (near-)deterministic outcome
    rows = []
    for c in df.columns.drop(target):
        if df[c].nunique() > max_levels: continue
        g = df.groupby(c)[target].agg(["mean", "size"]); g = g[g["size"] >= min_
        pure = g[(g["mean"] <= 0.01) | (g["mean"] >= 0.99)]
        if len(pure):
            rows.append({"attribute": c, "pure_groups": ", ".join(f"{k}→{m:.1%}
    return pd.DataFrame(rows)

suspects = leakage_suspects(raw, TARGET)
suspects.to_csv(REP / "leakage_suspects.csv", index=False)
print("\nLeakage suspects (groups with ≥99 % or ≤1 % cancellation):"); display(s

# Second detector that covers EVERY attribute, including high-cardinality (count
# how well does each attribute ALONE rank cancellations among later arrivals? Ca
```

```
# earlier arrivals only, and the evaluation block (60th-80th percentile of arriv
arrival = pd.to_datetime(raw["arrival_date_year"].astype(str) + "-" + raw["arriv
                         + raw["arrival_date_day_of_month"].astype(str), format=
fit_mask = arrival <= arrival.quantile(0.6)
eval_mask = (arrival > arrival.quantile(0.6)) & (arrival <= arrival.quantile(0.8
def single_feature_auc(col):
    s = raw[col]
    if s.dtype.kind in "if":
        score = s[eval_mask].fillna(s[fit_mask].median())
    else:
        enc = raw.loc[fit_mask, TARGET].groupby(s[fit_mask].astype(str)).mean()
        score = s[eval_mask].astype(str).map(enc).fillna(raw.loc[fit_mask, TARGE
    auc = roc_auc_score(raw.loc[eval_mask, TARGET], score)
    return max(auc, 1 - auc)
single_auc = pd.Series({c: single_feature_auc(c) for c in raw.columns.drop(TARGE
single_auc.to_frame("single_attribute_auc").to_csv(REP / "leakage_single_attribu
print("Single-attribute ROC-AUC on later arrivals (> 0.80 from one field alone i
display(single_auc.head(12).to_frame("single_attribute_auc"))
```

```
Exact duplicate rows                                    31994
Bookings with 0 guests (adults+children+babies)           180
Negative ADR                                                1
ADR > 1,000 EUR                                             1
ADR = 0 outside the Complementary segment                1279
Bookings with 0 nights                                    715
Stays longer than 30 nights                                24
More than 10 adults on one booking                         12
9 or more children or babies on one booking                 3
Meal = 'Undefined' (same as 'SC' per data paper)         1169
```

	missing	missing_%
children	4	0.00
country	488	0.41
agent	16340	13.69
company	112593	94.31

```
Leakage suspects (groups with ≥99 % or ≤1 % cancellation):
```

	attribute	pure_groups
0	deposit_type	Non Refund→99.4% (n=14,587.0)
1	required_car_parking_spaces	1→0.0% (n=7,383.0)
2	reservation_status	Canceled→100.0% (n=43,017.0), Check-Out→0.0% (...

```
Single-attribute ROC-AUC on later arrivals (> 0.80 from one field alone is suspic
ious):
```

	single_attribute_auc
reservation_status	1.000
lead_time	0.702
country	0.637
deposit_type	0.633
total_of_special_requests	0.612
market_segment	0.598
hotel	0.586
assigned_room_type	0.581
distribution_channel	0.579
customer_type	0.569
agent	0.567
booking_changes	0.565

Output: Data quality report

Problem	Extent	Remedy
`reservation_status`, `reservation_status_date`	perfectly reveal the outcome (100 % / 0 % cancellation by status; `reservation_status` alone gives AUC 1.00)	**exclude** (target leakage)
`assigned_room_type`	set at check-in	**exclude** (not known at booking time)
`booking_changes`	accumulates after booking	**exclude**
`required_car_parking_spaces`	0 cancellations among 7,416 bookings with parking → almost certainly recorded at arrival	**exclude**; sensitivity tested in 5.2
`country`	probably defaulted to Portugal for bookings that never reached check-in (2.3 finding 4)	**exclude**; sensitivity tested in 5.2
`total_of_special_requests`, `adr`	can be added (requests) or revised at checkout (ADR) after booking	keep: on their own they reach AUC 0.61 and well below 0.70 on later arrivals; dropping ADR changes nothing (5.2)
`deposit_type = Non Refund` cancels 99.4 %	14,587 bookings, mostly duplicated group/tour-operator blocks	keep; flagged as a question for the hotel; results reported with and without them (5.1)

Problem	Extent	Remedy
`company` missing	94.3 %	replace by a `has_company` flag (missing = no company)
`agent` missing	13.7 %	category `none` (missing = booked without an agent)
`children` missing	4 bookings	set to 0 (no children recorded), consistently for `children`, `total_guests` and `has_kids`
Exact duplicate rows	31,994 (27 %)	**keep**: there is no booking ID, and the profile (finding 8) shows they are mostly multi-room group bookings. Validation folds are cut at date boundaries so copies never straddle a fold (4.2)
0-guest bookings, negative ADR, ADR > €1,000	180, 1, 1	drop rows
ADR = 0 outside the Complementary segment	1,279	keep: free upgrades, staff and package stays are plausible and cannot be verified
0-night bookings	715	keep (day-use rooms are legitimate)
Stays > 30 nights; > 10 adults; ≥ 9 children or babies on one booking	24; 12; 3	keep: long stays and group bookings on one row are plausible, and 39 rows cannot move the model. The app accepts values up to twice the training maximum
`meal = Undefined`	1,169	recode as `SC` (no meal package), as defined in the data paper

What the leakage detectors can and cannot show.

- The first detector (groups with ≥ 99 % or ≤ 1 % cancellation) looks only at attributes with ≤ 30 levels.
- The second scores **every** attribute on its own on later arrivals. It is a screen for strong *single-field* leaks: only `reservation_status` stands out (AUC 1.00). It uses raw numeric values, so it only catches monotone patterns, and it cannot see a *partial* leak hidden inside a genuine field.

- That partial case is exactly what the within-segment test in 2.3 (finding 4) found for `country`.

🔍 Critic review — Data Understanding

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on CRISP-DM, a sought-after keynote speaker, and the author of several award-winning books on the CRISP-DM methodology. Review the **Data Understanding** phase of my hotel-cancellation project below. Check it against every generic task and output CRISP-DM 1.0 defines for this phase. Focus especially on: whether each hypothesis is testable, whether data quality issues are quantified, and whether every attribute was audited for availability at booking time (target leakage). List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Non Refund anomaly noted but not investigated; `country` may be corrected at check-in (possible leakage).	Added a Non Refund profile (97 % Portuguese group/tour-operator blocks); planned a sensitivity test for `country` (done in 5.2).
2	Leakage audit skipped high-cardinality fields; Non Refund and duplicate hypotheses untested; no timeline drift.	Single-attribute AUC screen over every field; NR is 98 % duplicates; duplicate profile; timeline plot; more quality rows.
3	The `country` defaulting hypothesis was testable but untested; drift comparison confounded by season.	Within-segment test: no-shows and cancellations ~95 % Portuguese vs ~25 % of check-outs → `country` excluded; like-for-like drift.

Phase 3 — Data Preparation

*Goal: construct the final modeling dataset. Outputs: **Dataset** and **Dataset description**.*

3.1 Select data

In [12]:
```
EXCLUDE = {
    "reservation_status": "encodes the outcome (Canceled / No-Show / Check-Out)
    "reservation_status_date": "date the outcome was recorded - known only after
    "assigned_room_type": "assigned at check-in - not known at booking time",
```

```
    "booking_changes": "accumulates after the booking is made - unknown at predi
    "required_car_parking_spaces": "no cancellations among bookings with parking
    "arrival_date_year": "calendar year does not generalise to future bookings",
    "arrival_date": "used only to order bookings for the time-based split",
    "country": "probably defaulted to Portugal for bookings that never reached c
}
TRANSFORMED = {"agent": "kept as the categorical agent_id (3.3)", "company": "94
               "total_of_special_requests": "AMBIGUOUS recording time (can be ad
               "adr": "AMBIGUOUS (can be revised at checkout); kept; sensitivity
selection_log = pd.DataFrame([{"attribute": c, "decision": "exclude" if c in EXC
                               "rationale": EXCLUDE.get(c, TRANSFORMED.get(c, "k
                              for c in list(raw.columns) + ["arrival_date"] if c
selection_log.to_csv(REP / "rationale_for_inclusion_exclusion.csv", index=False)
display(selection_log)
```

	attribute	decision	rationale
0	hotel	include	known at booking time, potentially predictive
1	lead_time	include	known at booking time, potentially predictive
2	arrival_date_year	exclude	calendar year does not generalise to future bo...
3	arrival_date_month	include	known at booking time, potentially predictive
4	arrival_date_week_number	include	known at booking time, potentially predictive
5	arrival_date_day_of_month	include	known at booking time, potentially predictive
6	stays_in_weekend_nights	include	known at booking time, potentially predictive
7	stays_in_week_nights	include	known at booking time, potentially predictive
8	adults	include	known at booking time, potentially predictive
9	children	include	known at booking time, potentially predictive
10	babies	include	known at booking time, potentially predictive
11	meal	include	known at booking time, potentially predictive
12	country	exclude	probably defaulted to Portugal for bookings th...
13	market_segment	include	known at booking time, potentially predictive
14	distribution_channel	include	known at booking time, potentially predictive
15	is_repeated_guest	include	known at booking time, potentially predictive
16	previous_cancellations	include	known at booking time, potentially predictive
17	previous_bookings_not_canceled	include	known at booking time, potentially predictive
18	reserved_room_type	include	known at booking time, potentially predictive
19	assigned_room_type	exclude	assigned at check-in - not known at booking time
20	booking_changes	exclude	accumulates after the booking is made - unknow...
21	deposit_type	include	known at booking time, potentially predictive
22	agent	include	kept as the categorical agent_id (3.3)
23	company	include	94 % missing - kept as the has_company flag (3.3)
24	days_in_waiting_list	include	known at booking time, potentially predictive
25	customer_type	include	known at booking time, potentially predictive
26	adr	include	AMBIGUOUS (can be revised at checkout); kept; ...
27	required_car_parking_spaces	exclude	no cancellations among bookings with parking -...

	attribute	decision	rationale
28	total_of_special_requests	include	AMBIGUOUS recording time (can be added after b...
29	reservation_status	exclude	encodes the outcome (Canceled / No-Show / Chec...
30	reservation_status_date	exclude	date the outcome was recorded - known only aft...
31	arrival_date	exclude	used only to order bookings for the time-based...

Output: Rationale for inclusion/exclusion

Attributes are excluded for one of three reasons:

- **they encode the outcome:** `reservation_status`, `reservation_status_date`, and `country`, which is probably defaulted for guests who never arrived (2.3 finding 4);
- **they are set or updated after booking:** `assigned_room_type`, `booking_changes`, `required_car_parking_spaces`;
- **they would not generalise:** `arrival_date_year`, because future bookings have unseen years.

`agent` and `company` are kept in their raw form and turned into `agent_id` and `has_company` by the score code in 3.3, so the model's input is exactly the **24 raw fields** a booking system exports. `total_of_special_requests` and `adr` have uncertain recording times. They are kept and labelled as such in the log, and their effect is measured in 5.2. Every other attribute is known at booking time and plausibly related to cancellation (2.3). Rows removed in 3.2 are invalid records (no guests, impossible prices): 182 bookings, 0.15 %.

3.2 Clean data

In [13]:
```
df = raw.copy(); log = []
def record(action, before, after):
    log.append({"action": action, "rows_before": before, "rows_after": after, "a

b = len(df); df = df[df[["adults", "children", "babies"]].fillna(0).sum(axis=1)
b = len(df); df = df[(df["adr"] >= 0) & (df["adr"] <= 1000)];
n_undef = int((df["meal"] == "Undefined").sum()); df["meal"] = df["meal"].replac
log.append({"action": "Meal 'Undefined' -> 'SC' (both = no meal package)", "rows
n_cty = int(df["country"].isna().sum()); df["country"] = df["country"].fillna("m
log.append({"action": "Missing country -> 'missing'", "rows_before": len(df), "r
log.append({"action": "Duplicates KEPT (no booking ID; likely group bookings)",
            "affected": int(df.duplicated().sum())})

cleaning_report = pd.DataFrame(log); cleaning_report.to_csv(REP / "data_cleaning
display(cleaning_report)
```

	action	rows_before	rows_after	affected
0	Drop bookings with 0 guests	119390	119210	180
1	Drop ADR < 0 or > 1,000 EUR	119210	119208	2
2	Meal 'Undefined' -> 'SC' (both = no meal package)	119208	119208	1169
3	Missing country -> 'missing'	119208	119208	478
4	Duplicates KEPT (no booking ID; likely group b...	119208	119208	31980

Output: Data cleaning report

Quality problem (2.4)	Action	Records affected	Possible impact
0-guest bookings	dropped	180	negligible (0.15 %)
ADR < 0 or > €1,000	dropped	2	negligible
`meal = Undefined`	recoded to `SC` (also inside the score code, for new data)	1,169	merges two codes with the same meaning
`country` missing	category `missing`	478	avoids inventing nationalities
duplicates	kept	31,980	must not leak across validation folds (handled in 4.2)
`children` missing	set to 0 inside the score code	4	negligible
implausible counts (very long stays, large parties, ADR = 0)	kept and documented (2.4)	1,318	small; plausible business cases

Final dataset: **119,208 bookings**. All imputation and encoding is fitted inside the model pipeline on training data only, so nothing is learned from the test period.

3.3 Construct data

In [14]:
```
# Construct data AS SCORE CODE: a small module shared by the notebook and the de
# booking fields straight from a property-management-system export and both appl
FEATURE_CODE = r'''
# Score code for the hotel-cancellation model: raw booking fields -> model input
import numpy as np
import pandas as pd

RAW_FIELDS = ["hotel", "lead_time", "arrival_date_month", "arrival_date_week_num
              "stays_in_weekend_nights", "stays_in_week_nights", "adults", "chil
              "market_segment", "distribution_channel", "is_repeated_guest", "pr
              "previous_bookings_not_canceled", "reserved_room_type", "deposit_t
              "days_in_waiting_list", "customer_type", "adr", "total_of_special_
RAW_CATEGORICAL = ["hotel", "arrival_date_month", "meal", "market_segment", "dis
                   "reserved_room_type", "deposit_type", "customer_type"]
```

```
NUMERIC = ["lead_time", "arrival_date_week_number", "arrival_date_day_of_month",
           "stays_in_week_nights", "adults", "children", "babies", "is_repeated_
           "previous_bookings_not_canceled", "days_in_waiting_list", "adr", "tot
           "total_nights", "total_guests", "has_kids", "has_company"]
CATEGORICAL = RAW_CATEGORICAL + ["agent_id"]


def prepare(raw):
    # Raw booking rows (RAW_FIELDS) -> model inputs. Stateless, so it is safe in
    d = pd.DataFrame(raw).copy()
    for c in RAW_FIELDS:
        if c not in RAW_CATEGORICAL:
            d[c] = pd.to_numeric(d[c], errors="coerce")        # 'NULL' agent/co
    d["children"] = d["children"].fillna(0)                      # 4 bookings: n
    d["meal"] = d["meal"].replace({"Undefined": "SC"})           # both mean 'no
    d["total_nights"] = d["stays_in_week_nights"] + d["stays_in_weekend_nights"]
    d["total_guests"] = d["adults"] + d["children"] + d["babies"]
    d["has_kids"] = ((d["children"] + d["babies"]) > 0).astype(int)
    d["agent_id"] = d["agent"].map(lambda a: "none" if pd.isna(a) else str(int(a
    d["has_company"] = d["company"].notna().astype(int)
    out = d[NUMERIC + CATEGORICAL].copy()
    out[CATEGORICAL] = out[CATEGORICAL].astype(object).where(out[CATEGORICAL].no
    return out


class CalibratedModel:
    # Deployed model: the evaluated pipeline's scores mapped through an isotonic
    # out-of-fold training predictions, so the output can be read (and summed) a
    def __init__(self, model, calibrator):
        self.model, self.calibrator = model, calibrator
        self.classes_ = np.array([0, 1])

    def predict_proba(self, X):
        p = np.clip(self.calibrator.predict(self.model.predict_proba(X)[:, 1]),
        return np.column_stack([1 - p, p])
'''
(APP / "booking_features.py").write_text(FEATURE_CODE.lstrip(), encoding="utf-8"
import sys, importlib
if str(APP) not in sys.path:
    sys.path.insert(0, str(APP))
import booking_features
importlib.reload(booking_features)
from booking_features import RAW_FIELDS, NUMERIC, CATEGORICAL, prepare, Calibrat

df["arrival_date"] = pd.to_datetime(df["arrival_date_year"].astype(str) + "-" +
                                    + df["arrival_date_day_of_month"].astype(str
derived = ["total_nights", "total_guests", "has_kids", "agent_id", "has_company"
display(prepare(df[RAW_FIELDS])[derived].describe(include="all").T[["count", "un
```

	count	unique	top	mean	min	max
total_nights	119208.0	NaN	NaN	3.426213	0.0	69.0
total_guests	119208.0	NaN	NaN	1.97121	1.0	55.0
has_kids	119208.0	NaN	NaN	0.078283	0.0	1.0
agent_id	119208	334	9	NaN	NaN	NaN
has_company	119208.0	NaN	NaN	0.056775	0.0	1.0

Output: Derived attributes

The construction step is written as **score code**, a small module `booking_features.py` saved next to the app. Its `prepare()` function is the first step of the model pipeline. The model therefore takes the **24 raw fields** a booking system exports and derives everything else itself, and the notebook and the app run exactly the same code.

Attribute	Definition	Rationale
`total_nights`, `total_guests`	sums	compact summaries of trip size. Their individual effect is not scored separately: 5.1 ranks the raw fields
`has_kids`	children or babies on the booking	a simple family-travel flag. No exploration finding tests it; kept because it is cheap and harmless for trees
`agent_id`	travel agent as a category (`none` if booked without one)	agents differ sharply (the 5 largest handle half of all bookings); `agent` is a top-4 driver in 5.1
`has_company`	a company pays	replaces the 94 %-missing `company` ID (2.4)

Removed in revision 2: `arrival_weekday`, `adr_per_guest`, `has_agent` and `room_type_x_hotel`. No exploration finding supported them, their permutation importances in the first run were 0.0021 or less, and `has_agent` duplicated `agent_id`.

Output: Generated records

None. Classes are only moderately imbalanced (≈ 37 % cancelled), so no oversampling.

3.4 Integrate data

In [15]:
```
# The two hotel files were harmonised and stacked in 2.2 (harmonize + concat). V
reference = pd.read_csv(COMBINED_REFERENCE)
checks = {
    "rows: resort + city = integrated":     (len(originals["Resort Hotel"]) + le
    "rows: integrated = published combined": (len(raw), len(reference)),
    "columns match published combined":     (sorted(raw.columns), sorted(referen
    "cancellations match":                  (int(raw[TARGET].sum()), int(referen
    "total ADR matches (rounded)":          (round(raw["adr"].sum()), round(refe
}
integration_report = pd.DataFrame([{"check": k, "passed": a == b} for k, (a, b)
integration_report.to_csv(REP / "integration_report.csv", index=False)
display(integration_report)
assert integration_report["passed"].all(), "Integration does not match the publi
```

	check	passed
0	rows: resort + city = integrated	True
1	rows: integrated = published combined	True
2	columns match published combined	True
3	cancellations match	True
4	total ADR matches (rounded)	True

Output: Merged data

Two source tables (resort, city) share one schema but were formatted inconsistently. Each was harmonised (snake_case names, trimmed text, `"NULL"` → missing, consistent types), given a `hotel` attribute to preserve provenance, and **stacked** (a union, not a join — there are no shared keys). The result was validated against the authors' published combined file: row count, columns, number of cancellations and total ADR all match exactly.

3.5 Format data

In [16]:
```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTransfo

# Time-based split: train on earlier arrivals, test on the most recent 20 % (mim
df = df.sort_values("arrival_date", kind="stable").reset_index(drop=True)
cutoff = df["arrival_date"].quantile(1 - TEST_SIZE)
is_test = df["arrival_date"] > cutoff

X = df[RAW_FIELDS]                          # raw booking fields; the pipeline's fi
y = df[TARGET].astype(int)
X_train, X_test, y_train, y_test = X[~is_test], X[is_test], y[~is_test], y[is_te

num_pipe = Pipeline([("impute", SimpleImputer(strategy="median")), ("scale", Sta
cat_pipe = Pipeline([("impute", SimpleImputer(strategy="constant", fill_value="m
                     ("onehot", OneHotEncoder(handle_unknown="infrequent_if_exis
                                              max_categories=30, sparse_output=F
preprocessor = ColumnTransformer([("num", num_pipe, NUMERIC), ("cat", cat_pipe,

dataset_description = pd.DataFrame({
    "set": ["train", "test"], "rows": [len(X_train), len(X_test)],
    "arrivals_from": [df.loc[~is_test, "arrival_date"].min().date(), df.loc[is_t
    "arrivals_to": [df.loc[~is_test, "arrival_date"].max().date(), df.loc[is_tes
    "cancel_rate": [round(y_train.mean(), 3), round(y_test.mean(), 3)]})
dataset_description.to_csv(REP / "dataset_description.csv", index=False)
print(f"{len(RAW_FIELDS)} raw booking fields in -> {len(NUMERIC)} numeric + {len
print(f"numeric: {NUMERIC}\ncategorical: {CATEGORICAL}"); display(dataset_descri
```

```
24 raw booking fields in -> 18 numeric + 9 categorical model inputs
numeric: ['lead_time', 'arrival_date_week_number', 'arrival_date_day_of_month',
'stays_in_weekend_nights', 'stays_in_week_nights', 'adults', 'children', 'babie
s', 'is_repeated_guest', 'previous_cancellations', 'previous_bookings_not_cancele
d', 'days_in_waiting_list', 'adr', 'total_of_special_requests', 'total_nights',
'total_guests', 'has_kids', 'has_company']
categorical: ['hotel', 'arrival_date_month', 'meal', 'market_segment', 'distribut
ion_channel', 'reserved_room_type', 'deposit_type', 'customer_type', 'agent_id']
```

	set	rows	arrivals_from	arrivals_to	cancel_rate
0	train	95413	2015-07-01	2017-04-22	0.361
1	test	23795	2017-04-23	2017-08-31	0.409

Output: Reformatted data · Dataset · Dataset description

The model takes **24 raw booking fields**. The score code turns them into **18 numeric and 9 categorical model inputs** (printed above). Numeric inputs are median-imputed and standardised; categorical inputs are one-hot encoded, with categories under 0.5 % of bookings (e.g. rare agents) pooled into an "infrequent" level so the model doesn't memorise them. Construction, imputation and encoding live in one scikit-learn pipeline that is fitted on training data only and saved with the model, so the app applies exactly the same transformations to raw booking-system data.

Some fields are exact sums of others (`total_nights` = week + weekend nights, `total_guests` = adults + children + babies). That is harmless for the tree models. For logistic regression the L2 penalty keeps the coefficients identified, but they should not be read one by one.

Time-based split: bookings are ordered by arrival date; the latest 20 % of arrivals (23 April – 31 August 2017, **23,795 bookings**, 40.9 % cancelled) form the test set, and everything earlier (1 July 2015 – 22 April 2017, **95,413 bookings**, 36.1 % cancelled) is for training. This mimics real use: the model is always applied to bookings that arrive after the data it learned from. Cancellations rose like for like (finding 7), so the test period is harder than the training period. 5.1 also checks a split by **booking** date, which is closer still to how the app is used.

🔍 Critic review — Data Preparation

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md` , `round_2.md` , `round_3.md` .

> *You are a world-renowned authority on CRISP-DM, a sought-after keynote speaker, and the author of several award-winning books on the CRISP-DM methodology. Review the **Data Preparation** phase of my hotel-cancellation project below. Check it against every generic task and output CRISP-DM 1.0 defines for this phase. Focus especially on: data leakage (both target leakage*

> *and train/test contamination), whether every quality issue from Phase 2 was resolved, and whether each derived attribute is justified by an exploration finding. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Keeping 27 % duplicate rows lets identical bookings sit in both training and validation folds.	Kept them (no booking ID) but changed the test design to time-series folds (4.2).
2	Booking date derivable; derived features unjustified (importance ≈ 0); folds could split same-day duplicates.	Four derived features dropped; construction moved into the pipeline as score code; date-aligned folds; booking-date check.
3	Selection log called ambiguous fields "known at booking"; a false rationale for derived features.	Ambiguous fields labelled as such; `country` excluded with evidence; rationales corrected.

Phase 4 — Modeling

Goal: select and apply modeling techniques and calibrate their parameters.

4.1 Select modeling technique

In [17]:
```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassif

class NonRefundRule(ClassifierMixin, BaseEstimator):
    """What a hotel could do without a model: treat every 'Non Refund' deposit b
    def fit(self, X, y):
        self.classes_ = np.array([0, 1]); return self
    def predict_proba(self, X):
        p = (pd.DataFrame(X)["deposit_type"] == "Non Refund").to_numpy(float)
        return np.column_stack([1 - p, p])
    def predict(self, X):
        return (self.predict_proba(X)[:, 1] >= 0.5).astype(int)

BASELINES = {   # benchmarks that use raw fields directly
    "Baseline: majority class": DummyClassifier(strategy="most_frequent"),
    "Baseline: Non Refund rule": NonRefundRule(),
    "Baseline: lead time only": Pipeline([("pick", ColumnTransformer([("lt", Fun
                                          ("model", LogisticRegression())]),
}
MODELS = {      # candidate models, each wrapped as prepare -> preprocess -> est
    "Logistic Regression": LogisticRegression(max_iter=3000),
    "Random Forest": RandomForestClassifier(n_estimators=150, min_samples_leaf=5
    "HistGradientBoosting": HistGradientBoostingClassifier(random_state=RANDOM_S
```

```
}
make_model = lambda est: Pipeline([("construct", FunctionTransformer(prepare)),
```

Output: Modeling technique

- **Baselines:** the majority class (predicts "not cancelled" for everyone: the floor), the **"Non Refund" rule** (flag every Non Refund deposit, which a hotel could do without any model), and **lead time only** (a one-variable logistic regression). A model is only worth deploying if it clearly beats the rule.
- **Logistic regression:** interpretable linear benchmark.
- **Random forest and histogram gradient boosting:** tree ensembles that capture interactions such as lead time × deposit type × segment without manual feature engineering.

No class weighting: the imbalance is mild (37 % positives). Probabilities are calibrated after model selection (5.1) and the decision threshold is tuned for business value.

Output: Modeling assumptions

Technique	Assumptions	How addressed
Logistic regression	linear effects in log-odds; scaled inputs; no exact collinearity	scaling in the pipeline; the exact sums (3.5) are handled by the L2 penalty, and coefficients are not interpreted one by one
Random forest / gradient boosting	no distributional assumptions; enough samples per leaf	`min_samples_leaf` ≥ 5 and depth capped — limits memorising duplicate rows and keeps the model small enough to deploy
All	the scores are not automatically calibrated probabilities	isotonic calibration on out-of-fold training scores; reliability curve and Brier score on test (5.1)
All	training and future bookings come from the same process	time-ordered validation and test; drift monitoring

4.2 Generate test design

In [18]:
```
from sklearn.model_selection import TimeSeriesSplit, cross_validate

# X_train is ordered by arrival date (3.5). Each fold trains on EARLIER arrivals
# exactly how the model will be used. Fold edges are moved to date boundaries so
# (27 % of rows are exact duplicates) can never sit on both sides of a fold. Shu
# it mixed past and future and over-estimated ROC-AUC (0.956 vs 0.868 on later a
def date_blocked_folds(dates, n_splits):
    dates = pd.Series(np.asarray(dates))
    starts = [dates.iloc[va[0]] for _, va in TimeSeriesSplit(n_splits=n_splits).
    ends = starts[1:] + [dates.iloc[-1] + pd.Timedelta(days=1)]
    return [(np.flatnonzero(dates < s), np.flatnonzero((dates >= s) & (dates < e

train_dates = df.loc[X_train.index, "arrival_date"].reset_index(drop=True)
CV = date_blocked_folds(train_dates, CV_FOLDS)
SCORING = {"roc_auc": "roc_auc", "precision": "precision", "recall": "recall", "
```

```
PRIMARY = "roc_auc"
FOLD_NAMES = []
for i, (tr, va) in enumerate(CV, 1):
    FOLD_NAMES.append(f"fold {i}: {train_dates.iloc[va].min():%Y-%m-%d}..{train_
    print(f"fold {i}: train {train_dates.iloc[tr].min().date()} .. {train_dates.
          f"  ->  validate {train_dates.iloc[va].min().date()} .. {train_dates.i
```

```
fold 1: train 2015-07-01 .. 2015-10-24 (15,851)  ->  validate 2015-10-25 .. 2016-
03-24 (15,936)
fold 2: train 2015-07-01 .. 2016-03-24 (31,787)  ->  validate 2016-03-25 .. 2016-
06-21 (15,902)
fold 3: train 2015-07-01 .. 2016-06-21 (47,689)  ->  validate 2016-06-22 .. 2016-
09-27 (15,820)
fold 4: train 2015-07-01 .. 2016-09-27 (63,509)  ->  validate 2016-09-28 .. 2017-
01-07 (15,898)
fold 5: train 2015-07-01 .. 2017-01-07 (79,407)  ->  validate 2017-01-08 .. 2017-
04-22 (16,006)
```

Output: Test design

*Model selection uses **time-series cross-validation** (5 expanding-window folds, printed above) on the **training period only**: every fold trains on earlier arrivals and validates on later ones, mirroring real use. Fold edges are moved to **date boundaries**, so the 27 % exact-duplicate rows (mostly multi-room group bookings, 2.3 finding 8) can never sit on both sides of a fold. Shuffled K-fold was rejected because it mixes future bookings into training and puts identical rows on both sides of a fold. That inflated ROC-AUC well above what the model achieves on genuinely new bookings (4.4). Tuning uses the same folds, so tuned and default scores are directly comparable, and per-fold scores are reported so differences can be judged against fold-to-fold noise.*

*Honest note on the test period: in the first iteration it was scored once and compared with shuffled CV, and that gap (0.956 vs 0.868) led to this design. Its results were also visible during the two later review rounds. Every later change was justified on cross-validation and on data-understanding evidence (for example the nationality test), but the test period is no longer pristine. **Cross-validation is therefore reported as primary evidence next to every test result** (5.1). Primary metric ROC-AUC (threshold-free ranking quality); precision and recall are evaluated at the business-optimal threshold chosen on training data.*

4.3 Build model

In [19]:
```
rows, fold_auc = [], {}
for name, est in {**BASELINES, **{m: make_model(e) for m, e in MODELS.items()}}.
    r = cross_validate(est, X_train, y_train, cv=CV, scoring=SCORING, n_jobs=-1)
    fold_auc[name] = r[f"test_{PRIMARY}"]
    rows.append({"model": name, **{f"{m}_mean": r[f"test_{m}"].mean() for m in S
                 f"{PRIMARY}_std": r[f"test_{PRIMARY}"].std(), "fit_time_s": r["
    print(f"{name:<28} ROC-AUC {rows[-1]['roc_auc_mean']:.4f}")
cv_table = pd.DataFrame(rows).set_index("model").round(4).sort_values(f"{PRIMARY
cv_table.to_csv(REP / "model_cv_comparison.csv"); display(cv_table)
print("ROC-AUC per fold (default settings):")
per_fold = pd.DataFrame(fold_auc, index=FOLD_NAMES).T.round(4); per_fold.to_csv(
```

```
Baseline: majority class     ROC-AUC 0.5000
Baseline: Non Refund rule    ROC-AUC 0.6643
```

```
Baseline: lead time only      ROC-AUC 0.6878
Logistic Regression           ROC-AUC 0.8117
Random Forest                 ROC-AUC 0.8442
HistGradientBoosting          ROC-AUC 0.8401
```

model	roc_auc_mean	precision_mean	recall_mean	f1_mean	accuracy_mea
Random Forest	0.8442	0.8954	0.4687	0.6099	0.791
HistGradientBoosting	0.8401	0.8211	0.5404	0.6420	0.793
Logistic Regression	0.8117	0.7900	0.5415	0.5910	0.734
Baseline: lead time only	0.6878	0.5336	0.3889	0.4236	0.649
Baseline: Non Refund rule	0.6643	0.9922	0.3304	0.4917	0.762
Baseline: majority class	0.5000	0.0000	0.0000	0.0000	0.647

ROC-AUC per fold (default settings):

	fold 1: 2015-10-25..2016-03-24	fold 2: 2016-03-25..2016-06-21	fold 3: 2016-06-22..2016-09-27	fold 4: 2016-09-28..2017-01-07	fold 5: 2017-01-08..2017-04-22
Baseline: majority class	0.5000	0.5000	0.5000	0.5000	0.5000
Baseline: Non Refund rule	0.6871	0.7120	0.6160	0.6490	0.6577
Baseline: lead time only	0.7369	0.6676	0.6183	0.7229	0.6934
Logistic Regression	0.7500	0.8296	0.8073	0.8212	0.8505
Random Forest	0.8439	0.8618	0.8098	0.8422	0.8633
HistGradientBoosting	0.8352	0.8568	0.8058	0.8350	0.8677

In [20]:
```
from sklearn.model_selection import RandomizedSearchCV
SEARCH_SPACES = {
    "Logistic Regression": {"C": loguniform(1e-3, 1e2)},
    "Random Forest": {"n_estimators": [100, 200], "max_depth": [15, 25, 40], "mi
                      "max_features": ["sqrt", 0.3]},
    "HistGradientBoosting": {"learning_rate": loguniform(0.03, 0.3), "max_iter":
                             "max_leaf_nodes": [31, 63, 127], "min_samples_leaf"
                             "l2_regularization": loguniform(1e-4, 1.0)},
}
# Every family gets the same random-search budget on the same folds. Tuned setti
# defaults by more than one standard error of the paired per-fold differences (a
tuned, tuning_rows, tuning_logs = {}, [], []
for name, est in MODELS.items():
    search = RandomizedSearchCV(make_model(est), {f"model__{k}": v for k, v in S
                                n_iter=N_TUNING_ITER, cv=CV, scoring=PRIMARY, n_
    search.fit(X_train, y_train)
    best_folds = np.array([search.cv_results_[f"split{i}_test_score"][search.bes
```

```
    diff = best_folds - fold_auc[name]; se = diff.std(ddof=1) / np.sqrt(len(diff
    adopt = diff.mean() > se
    params = {k.replace("model__", ""): (round(v, 5) if isinstance(v, float) els
    tuned[name] = {"fold_auc": best_folds if adopt else fold_auc[name], "params"
                   "estimator": search.best_estimator_ if adopt else make_model(
    tuning_rows.append({"model": name, "default_auc": fold_auc[name].mean(), "be
                        "gain": diff.mean(), "gain_se": se, "adopted": "tuned" i
    tuning_logs.append(pd.DataFrame(search.cv_results_)[["params", "mean_test_sc
tuning_table = pd.DataFrame(tuning_rows).set_index("model").round(4)
pd.concat(tuning_logs).sort_values(["model", "mean_test_score"], ascending=[True
display(tuning_table)

# Champion = best mean over the same folds; show whether its lead is consistent
means = {n: t["fold_auc"].mean() for n, t in tuned.items()}
best_name = max(means, key=means.get)
paired = pd.DataFrame([{"champion": best_name, "versus": other, "mean_auc_gain":
                        "se": (tuned[best_name]["fold_auc"] - t["fold_auc"]).std
                        "folds_won": f"{int(((tuned[best_name]['fold_auc'] - t['
                       for other, t in tuned.items() if other != best_name]).rou
paired.to_csv(REP / "champion_paired_comparison.csv", index=False); display(pair
best_model, best_params = tuned[best_name]["estimator"], tuned[best_name]["param
print(f"Champion: {best_name} ({best_params}), mean CV ROC-AUC {means[best_name]
```

model	default_auc	best_tuned_auc	gain	gain_se	adopted	s
Logistic Regression	0.8117	0.8159	0.0042	0.0044	defaults	np.float64(0.
Random Forest	0.8442	0.8489	0.0046	0.0052	defaults	{'n_estimato 'min_sampl
HistGradientBoosting	0.8401	0.8340	-0.0061	0.0022	defaults	{'l2_regular np.float64(0

	champion	versus	mean_auc_gain	se	folds_won
0	Random Forest	Logistic Regression	0.0325	0.0161	5/5
1	Random Forest	HistGradientBoosting	0.0041	0.0023	4/5

```
Champion: Random Forest (defaults), mean CV ROC-AUC 0.8442
```

Output: Parameter settings

Every model family got the **same tuning budget**: 8 random-search candidates on the same five folds. Tuned settings were adopted only if they beat the defaults by more than one standard error of the paired per-fold differences, because the best of 8 noisy scores is optimistic by construction. Eight candidates is a thin budget for the 5-parameter gradient-boosting space, so its ranking could change with a larger search.

Family	Default CV ROC-AUC	Best tuned	Adopted
Random forest	0.844	0.849 (+0.005 ± 0.005)	**defaults**: the gain is within noise
Gradient boosting	0.840	0.834	defaults (tuning did not help)
Logistic regression	0.812	0.816 (+0.004 ± 0.004)	defaults: the gain is within noise

Final: random forest with 150 trees, max depth 25, at least 5 bookings per leaf, √p features per split. It beats logistic regression on **5 of 5 folds** (+0.033 ± 0.016 AUC) and gradient boosting on **4 of 5 folds** (+0.004 ± 0.002). Its lead over boosting is consistent but small. The full tuning log is in `artifacts/reports/tuning_results.csv` and the paired comparison in `champion_paired_comparison.csv`.

Output: Models

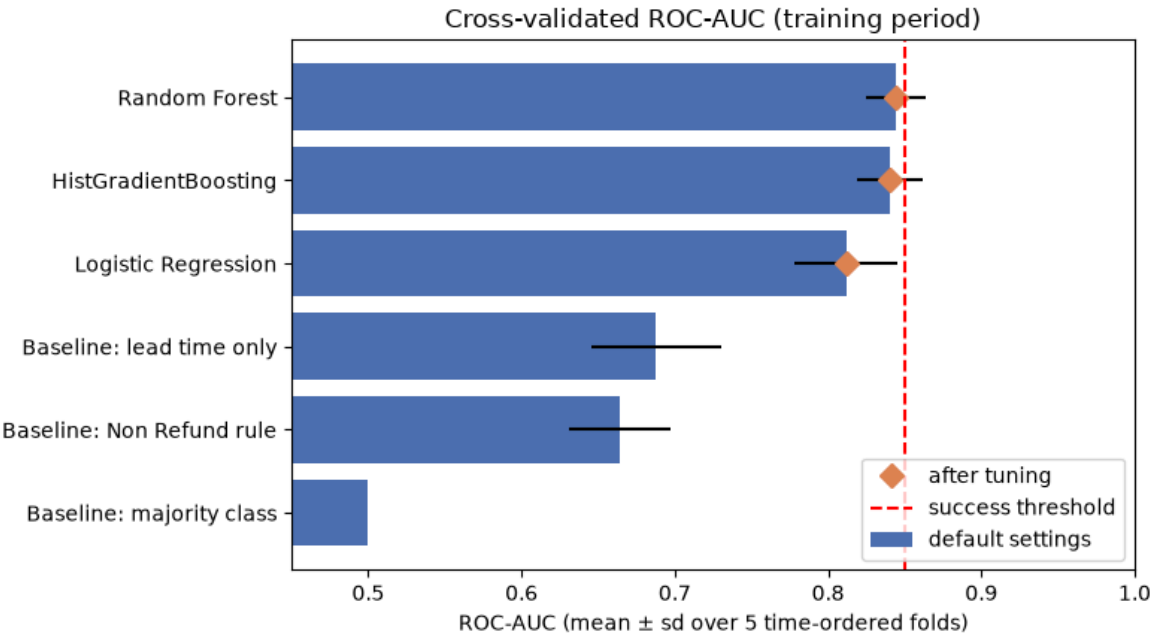
Three baselines, three candidate pipelines (score code + preprocessing + estimator) and their tuned variants.

Output: Model description

The approved model averages 150 decision trees. Each is grown on a bootstrap sample of the training bookings, using a random subset of attributes at each split, with at least 5 bookings per leaf. For a new booking, each tree returns the share of cancelled training bookings in the leaf it lands in, and the forest averages these shares. That score ranks bookings but is not a calibrated probability, so 5.1 calibrates it. It relies most on deposit type, special requests, lead time, travel agent and customer type (permutation importance, 5.1).

4.4 Assess model

In [21]:
```
fig, ax = plt.subplots(figsize=(8, 4.5))
plot_df = cv_table.iloc[::-1]
ax.barh(plot_df.index, plot_df["roc_auc_mean"], xerr=plot_df["roc_auc_std"], col
for n, t in tuned.items():
    ax.plot(t["fold_auc"].mean(), n, "D", c="#DD8452", ms=8, label="after tuning
ax.axvline(SUCCESS_CRITERIA["roc_auc"], ls="--", c="red", label="success thresho
ax.set_xlim(0.45, 1); ax.set_xlabel("ROC-AUC (mean ± sd over 5 time-ordered fold
ax.set_title("Cross-validated ROC-AUC (training period)"); ax.legend(loc="lower
```



Output: Model assessment

Model	CV ROC-AUC (mean ± sd)	Comment
Random forest	**0.844 ± 0.019**	best on 4–5 of 5 folds against each rival; adopted
Gradient boosting	0.840 ± 0.021	close second
Logistic regression	0.812 ± 0.034	linear benchmark, clearly behind once nationality is removed
Baseline: lead time only	0.688	the single strongest attribute on its own
Baseline: Non Refund rule	0.664	precision 0.99 but recall only 0.33: it finds one kind of cancellation
Baseline: majority class	0.500	floor

Without nationality, no model reaches the 0.85 ROC-AUC criterion on cross-validation (the random forest comes closest at 0.844). With nationality it was 0.878, and 5.2 shows what that field was worth. All three models beat the best baseline by 0.12–0.16. The hardest block is summer 2016 (fold 3: RF 0.810), and the easiest are spring 2016 and early 2017 (folds 2 and 5: 0.862–0.863), so difficulty varies with the season. `min_samples_leaf` = 5 keeps the trees from fitting single bookings. It cannot stop a set of dozens of identical duplicate rows from filling a leaf, which is why the folds are cut at date boundaries.

Output: Revised parameter settings

- **Iteration 1 → 2:** with shuffled 5-fold CV the random forest scored ROC-AUC 0.956, but only 0.868 on the later test period. The test design was changed to date-aligned time-series folds.

- **Iteration 2 → 3:** three baselines, an equal tuning budget and a one-standard-error adoption rule were added; the defaults were kept.
- **Iteration 3 → 4:** `country` was removed after the defaulting test (2.3). CV ROC-AUC fell from 0.878 to 0.844.

🔍 Critic review — Modeling

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on CRISP-DM, a sought-after keynote speaker, and the author of several award-winning books on the CRISP-DM methodology. Review the **Modeling** phase of my hotel-cancellation project below. Check it against every generic task and output CRISP-DM 1.0 defines for this phase. Focus especially on: whether the baseline and benchmark comparisons are meaningful, whether the test design prevents optimistic bias (time-based hold-out, no tuning on test), whether the search space is sensible, and whether model assumptions were checked. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Shuffled CV gave 0.956 vs 0.868 on the later test period; tuned settings did not beat the defaults.	Time-series CV (now 0.877 vs 0.868); tuning adopted only if it beats defaults on the same folds.
2	Three figures corrupted by a plotting bug; test period informed the redesign; no rule baseline; unequal tuning.	Figures fixed; Non Refund and lead-time baselines; equal tuning with a 1-SE adoption rule; paired per-fold comparison; disclosure.
3	Test results visible during every revision; calibrator fitted on weaker fold models; thin search budget for boosting.	CV reported as primary evidence; calibrator on folds 3–5; budget limit acknowledged. Without `country`: CV AUC 0.844.

Phase 5 — Evaluation

*Goal: judge the model against the **business** objectives, review the process, and decide the next steps.*

5.1 Evaluate results

In [22]:
```
from sklearn import metrics as M
from sklearn.base import clone
from sklearn.isotonic import IsotonicRegression

# 1) Out-of-fold scores for every training validation block (training data only)
def oof_scores(model):
    s = np.full(len(y_train), np.nan)
    for tr, va in CV:
        s[va] = clone(model).fit(X_train.iloc[tr], y_train.iloc[tr]).predict_pro
    return s
oof = oof_scores(best_model)
has = ~np.isnan(oof); y_oof, oof_raw = y_train.to_numpy()[has], oof[has]

# 2) Calibrate: isotonic map from score to observed cancellation rate, fitted on
late = np.zeros(len(y_train), bool); late[np.concatenate([va for _, va in CV[2:]
calibrator = IsotonicRegression(out_of_bounds="clip", y_min=0, y_max=1).fit(oof_
oof_cal = calibrator.predict(oof_raw)
deployed_model = CalibratedModel(best_model, calibrator)

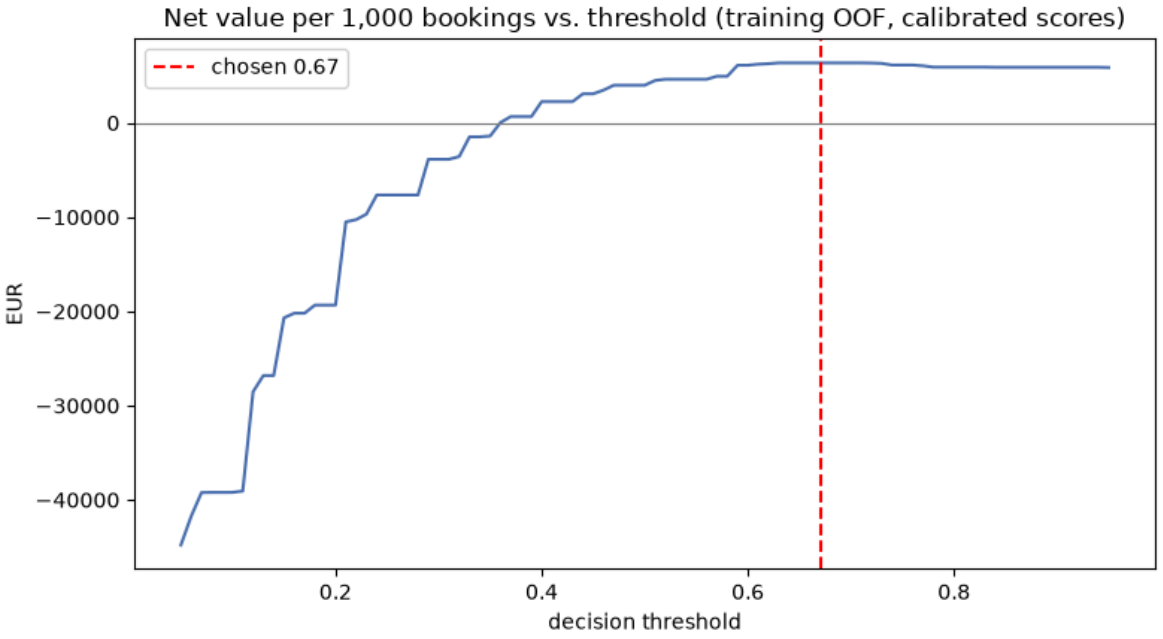
# 3) Choose the decision threshold on the calibrated training OOF scores, maximi
def net_value(y_true, p, t, cost_fp=COST_FP, value_tp=VALUE_TP):
    yp = p >= t; yt = np.asarray(y_true) == 1
    return (yp & yt).sum() * value_tp - (yp & ~yt).sum() * cost_fp - (~yp & yt).
grid = np.round(np.arange(0.05, 0.96, 0.01), 2)
train_value = np.array([net_value(y_oof, oof_cal, t) for t in grid])
DECISION_THRESHOLD = float(grid[train_value.argmax()])
oof_flag = oof_cal >= DECISION_THRESHOLD
print(f"Decision threshold (calibrated probability, chosen on training OOF): {DE
      f"training OOF precision {M.precision_score(y_oof, oof_flag):.3f}, recall
fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(grid, train_value / len(y_oof) * 1000, c="#4C72B0"); ax.axvline(DECISION
ax.axhline(0, c="gray", lw=.8); ax.legend()
ax.set(title="Net value per 1,000 bookings vs. threshold (training OOF, calibrat
savefig("eval_threshold_selection")

# 4) Evaluate on the held-out test period (ROC-AUC on the raw ranking scores; fl
proba_raw = best_model.predict_proba(X_test)[:, 1]
proba = calibrator.predict(proba_raw)
pred = (proba >= DECISION_THRESHOLD).astype(int)
test_metrics = {"roc_auc": M.roc_auc_score(y_test, proba_raw), "pr_auc": M.avera
                "precision": M.precision_score(y_test, pred), "recall": M.recall
                "f1": M.f1_score(y_test, pred), "accuracy": M.accuracy_score(y_t
                "brier_raw": M.brier_score_loss(y_test, proba_raw), "brier_calib

# 95 % confidence intervals: block bootstrap over arrival weeks (bookings in the
weeks = df.loc[X_test.index, "arrival_date"].dt.to_period("W").to_numpy()
week_rows = {w: np.flatnonzero(weeks == w) for w in np.unique(weeks)}
rng, yt_all, boots = np.random.default_rng(RANDOM_STATE), y_test.to_numpy(), []
for _ in range(300):
    ix = np.concatenate([week_rows[w] for w in rng.choice(list(week_rows), len(w
    boots.append({"roc_auc": M.roc_auc_score(yt_all[ix], proba_raw[ix]), "precis
                  "recall": M.recall_score(yt_all[ix], pred[ix])})
ci = pd.DataFrame(boots).quantile([0.025, 0.975])
crit = pd.DataFrame([{"criterion": m, "threshold": t, "test_value": round(test_m
                      "ci_95_low": round(ci.loc[0.025, m], 4), "ci_95_high": rou
                      "met": test_metrics[m] >= t, "met_at_ci_low": ci.loc[0.025
crit.to_csv(REP / "success_criteria_assessment.csv", index=False)
cv_crit = {"roc_auc": M.roc_auc_score(y_oof, oof_raw), "precision": M.precision_
crit.insert(2, "cv_value", [round(cv_crit[m], 4) for m in crit["criterion"]])
```

```
crit.to_csv(REP / "success_criteria_assessment.csv", index=False)
print({k: round(v, 4) for k, v in test_metrics.items()}); display(crit)
print(M.classification_report(y_test, pred, target_names=["not cancelled", "canc
```

```
Decision threshold (calibrated probability, chosen on training OOF): 0.67 | train
ing OOF precision 0.949, recall 0.405
```




```
{'roc_auc': 0.8269, 'pr_auc': 0.7961, 'precision': 0.8314, 'recall': 0.4709, 'f
1': 0.6012, 'accuracy': 0.7447, 'brier_raw': 0.168, 'brier_calibrated': 0.1632}
```

	criterion	threshold	cv_value	test_value	ci_95_low	ci_95_high	met	met_at_ci_low
0	roc_auc	0.85	0.8412	0.8269	0.8096	0.8461	False	False
1	precision	0.75	0.9491	0.8314	0.7972	0.8579	True	True
2	recall	0.50	0.4045	0.4709	0.4220	0.5245	False	False

```
              precision    recall  f1-score   support

not cancelled      0.719     0.934     0.812     14069
    cancelled      0.831     0.471     0.601      9726

     accuracy                          0.745     23795
    macro avg      0.775     0.702     0.707     23795
 weighted avg      0.765     0.745     0.726     23795
```

In [23]:

```
from sklearn.calibration import calibration_curve
fig, axes = plt.subplots(1, 4, figsize=(21, 4.5))
M.ConfusionMatrixDisplay.from_predictions(y_test, pred, ax=axes[0], colorbar=Fal
axes[0].set_title(f"Confusion matrix (threshold {DECISION_THRESHOLD:.2f})")
M.RocCurveDisplay.from_predictions(y_test, proba_raw, ax=axes[1]); axes[1].set_t
M.PrecisionRecallDisplay.from_predictions(y_test, proba_raw, ax=axes[2]); axes[2
for p, lab in [(proba_raw, "raw random-forest score"), (proba, "after isotonic c
    fp, mp = calibration_curve(y_test, p, n_bins=10, strategy="uniform"); axes[3
axes[3].plot([0, 1], [0, 1], "k--", lw=1); axes[3].legend(); axes[3].set(title="
savefig("eval_test_performance")

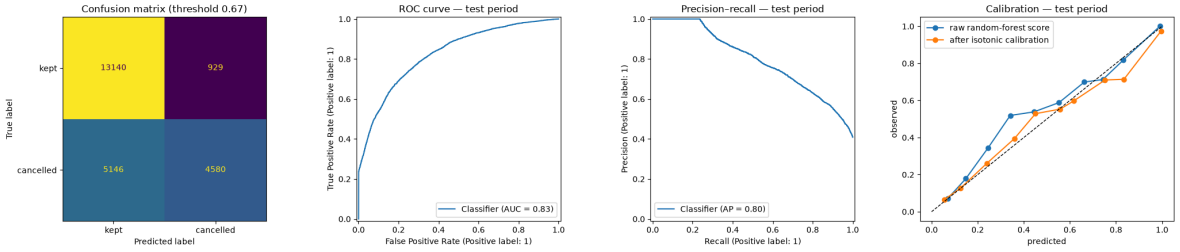
# Robustness: does it work equally well for both hotels?
by_hotel = pd.DataFrame([{"hotel": h, "bookings": int(m.sum()), "cancel_rate": r
                          "roc_auc": round(M.roc_auc_score(y_test[m], proba_raw[
                          "precision": round(M.precision_score(y_test[m], pred[m
```

```
                            "recall": round(M.recall_score(y_test[m], pred[m]), 4)
                           for h in X_test["hotel"].unique() for m in [(X_test["ho
by_hotel.to_csv(REP / "performance_by_hotel.csv", index=False); display(by_hotel

# Robustness: the app scores NEW bookings, so split the test period by booking d
# Bookings made after the training cut-off are exactly the ones the deployed mod
booking_date = df["arrival_date"] - pd.to_timedelta(df["lead_time"], unit="D")
made_after = (booking_date.loc[X_test.index] > cutoff).to_numpy()
by_booking = pd.DataFrame([{"test bookings": lab, "bookings": int(m.sum()), "can
                            "median_lead_time": int(X_test["lead_time"][m].media
                            "roc_auc": round(M.roc_auc_score(y_test[m], proba_ra
                            "precision": round(M.precision_score(y_test[m], pred
                           for lab, m in [("booked before the cut-off", ~made_af
by_booking.to_csv(REP / "performance_by_booking_date.csv", index=False); display

# The after-cut-off group is truncated to short lead times (the data ends in Aug
band = pd.cut(X_test["lead_time"], [-1, 7, 30, 90, 130], labels=["0-7", "8-30",
rows = []
for b in ["0-7", "8-30", "31-90", "91-130"]:
    for lab, m in [("before", ~made_after), ("after", made_after)]:
        mm = (band == b) & m
        if mm.sum() > 200 and 0 < y_test[mm].mean() < 1:
            rows.append({"lead_time_days": b, "booked": lab, "bookings": int(mm.
                         "roc_auc": M.roc_auc_score(y_test[mm], proba_raw[mm])})
display(pd.DataFrame(rows).round(3).pivot(index="lead_time_days", columns="booke

# How much of the result comes from the Non Refund blocks (a suspected data arte
nr_t = (X_test["deposit_type"] == "Non Refund").to_numpy()
sub = pd.DataFrame([{"test bookings": lab, "bookings": int(m.sum()), "cancel_rat
                     "roc_auc": round(M.roc_auc_score(y_test[m], proba_raw[m]),
                     "recall": round(M.recall_score(y_test[m], pred[m]), 4)} for
sub.to_csv(REP / "performance_without_non_refund.csv", index=False); display(sub
```



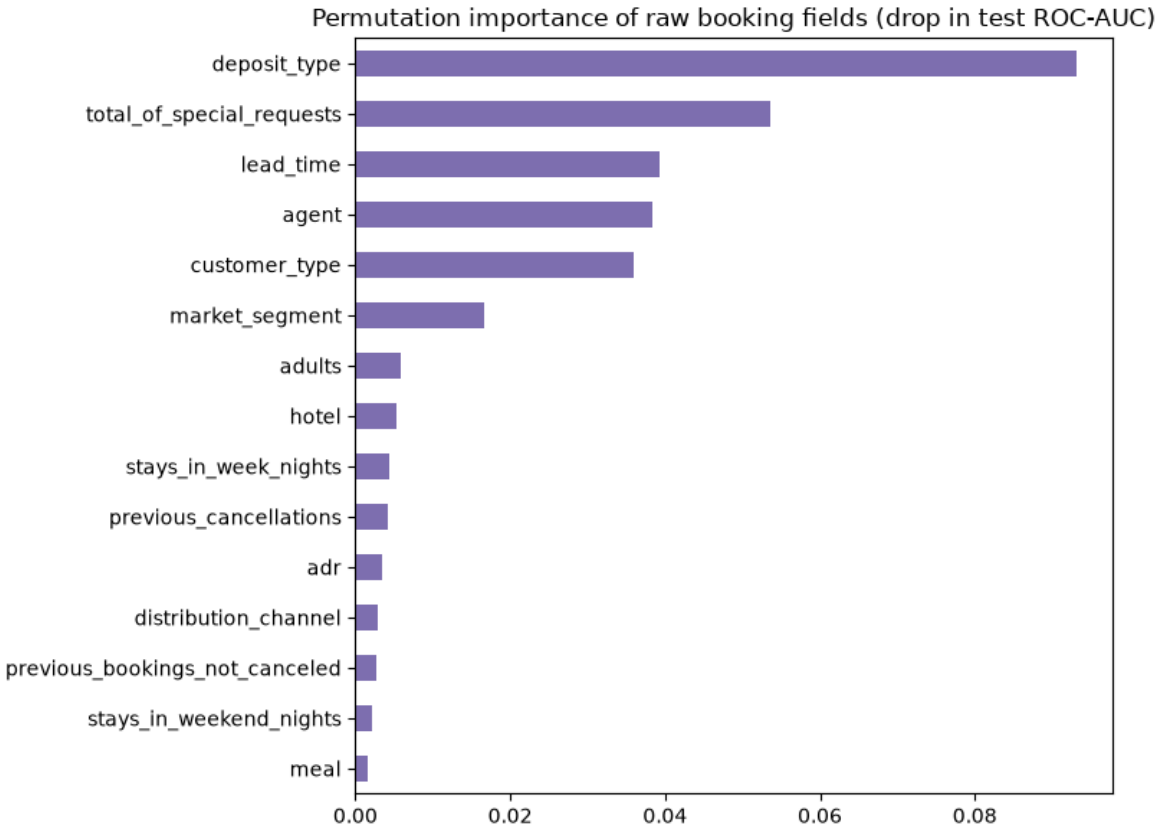
	hotel	bookings	cancel_rate	roc_auc	precision	recall
0	Resort Hotel	7459	0.357	0.8054	0.9201	0.2161
1	City Hotel	16336	0.432	0.8371	0.8200	0.5671

	test bookings	bookings	cancel_rate	median_lead_time	roc_auc	precision	recall
0	booked before the cut-off	16370	0.470	166	0.8401	0.8419	0.5551
1	booked after the cut-off	7425	0.274	19	0.7602	0.7087	0.1521

	bookings		cancel_rate		roc_auc	
booked	after	before	after	before	after	before
lead_time_days						
0-7	2341.0	NaN	0.109	NaN	0.680	NaN
31-90	2185.0	1810.0	0.403	0.382	0.687	0.789
8-30	2500.0	511.0	0.293	0.391	0.715	0.730
91-130	399.0	2845.0	0.411	0.433	0.781	0.801

	test bookings	bookings	cancel_rate	roc_auc	precision	recall
0	Non Refund	2291	1.000	1.0000	1.0000	0.9996
1	all other bookings	21504	0.346	0.7737	0.7115	0.3081

In [24]:
```
from sklearn.inspection import permutation_importance
pi_rows = X_test.sample(min(8000, len(X_test)), random_state=RANDOM_STATE).index
pi = permutation_importance(best_model, X_test.loc[pi_rows], y_test.loc[pi_rows]
                            random_state=RANDOM_STATE)   # single process: avoid
importance = pd.Series(pi.importances_mean, index=X_test.columns).sort_values()
fig, ax = plt.subplots(figsize=(8, 6))
importance.tail(15).plot.barh(ax=ax, color="#8172B2", title="Permutation importa
savefig("eval_feature_importance")
importance.sort_values(ascending=False).round(4).to_csv(REP / "feature_importanc
display(importance.sort_values(ascending=False).head(8).round(4).to_frame("drop_
```



	drop_in_roc_auc
deposit_type	0.0931
total_of_special_requests	0.0536
lead_time	0.0394
agent	0.0384
customer_type	0.0359
market_segment	0.0166
adults	0.0058
hotel	0.0054

In [25]:
```
# Business impact on the test period.
# (a) Per booking: what each flagging strategy is worth under the cost model (th
per_k = lambda v: v / len(y_test) * 1000
nr_flag = (X_test["deposit_type"] == "Non Refund").to_numpy()
lr_model = tuned["Logistic Regression"]["estimator"]
lr_oof = oof_scores(lr_model)[has]
lr_t = float(grid[np.argmax([net_value(y_oof, lr_oof, t) for t in grid])])
lr_proba = lr_model.predict_proba(X_test)[:, 1]
strategies = {"Status quo (no model)": np.zeros(len(y_test), bool), "Flag every
              "Rule: flag Non Refund deposits": nr_flag, f"Logistic regression (
              f"{best_name}, threshold 0.50": proba >= 0.5, f"{best_name}, thres
business = pd.DataFrame([{"strategy": s, "net_value_eur_per_1000": per_k(net_val
                          "bookings_flagged_%": f.mean() * 100, "precision": M.p
                          "recall": M.recall_score(y_test, f)} for s, f in strat
business.to_csv(REP / "business_impact.csv", index=False); display(business)

# Sensitivity to the cost assumption (wrong-flag cost / right-flag value); thres
sens = []
for ratio in [0.5, 1, 2, 3, 4]:
    cfp = ratio * VALUE_TP
    t = float(grid[np.argmax([net_value(y_oof, oof_cal, g, cost_fp=cfp) for g in
    f = proba >= t
    sens.append({"cost_FP / value_TP": ratio, "threshold": t, "model_eur_per_100
                 "non_refund_rule_eur_per_1000": per_k(net_value(y_test, nr_flag
                 "flagged_%": f.mean() * 100, "precision": M.precision_score(y_t
cost_sensitivity = pd.DataFrame(sens).round(3); cost_sensitivity.to_csv(REP / "c
display(cost_sensitivity)

# (b) Per NIGHT: the real decision is how many extra rooms to sell for each arri
# false flags on the same night offset each other, so the value is simulated per
#   re-sold rooms = min(allowance, actual cancellations); walked guests = max(0,
#   both on a full night only (x P_FULL). The INCUMBENT is the hotel's rule of t
#   of that hotel and arrival month (training period) x bookings on the books.
t_df = df.loc[X_test.index, ["hotel", "arrival_date"]].assign(y=y_test.to_numpy(
hist = df.loc[~is_test].groupby(["hotel", df.loc[~is_test, "arrival_date"].dt.mo
t_df["hist"] = [hist.get((h, m), y_train.mean()) for h, m in zip(t_df["hotel"],
nights = t_df.groupby(["hotel", "arrival_date"]).agg(bookings=("y", "size"), can
                                                     flags=("flag", "sum"), nr=(
night_value = lambda allow, c: P_FULL * (RESALE_VALUE * np.minimum(allow, c) - W
POLICIES = {"No overbooking": lambda n: np.zeros(len(n)),
            "Incumbent: historical rate x bookings": lambda n: np.round(n["hist"
            "Rule: count Non Refund bookings": lambda n: n["nr"],
```

```
            f"Model: count flagged bookings (p >= {DECISION_THRESHOLD:.2f})": la
            "Model: sum of probabilities (expected cancellations)": lambda n: np
for name, pol in POLICIES.items():
    nights[name] = night_value(pol(nights), nights["cancels"])
weeks_n = nights["arrival_date"].dt.to_period("W").astype(str).to_numpy()
n_weeks = len(np.unique(weeks_n))
boot_idx = np.random.default_rng(RANDOM_STATE).integers(0, n_weeks, (500, n_week
def boot_ci(col):
    s = nights.groupby(weeks_n)[[col, "bookings"]].sum()
    a, b = s[col].to_numpy()[boot_idx].sum(1), s["bookings"].to_numpy()[boot_idx
    return np.quantile(a / b * 1000, [.025, .975])
inc = "Incumbent: historical rate x bookings"
per_night = []
for name, pol in POLICIES.items():
    allow = pol(nights)
    nights[f"diff_{name}"] = nights[name] - nights[inc]
    lo, hi = boot_ci(f"diff_{name}") if name != inc else (0.0, 0.0)
    per_night.append({"policy": name, "eur_per_1000_bookings": nights[name].sum(
                      "vs_incumbent": nights[f"diff_{name}"].sum() / nights["boo
                      "mean_abs_error_rooms_per_night": np.mean(np.abs(allow - n
per_night = pd.DataFrame(per_night).set_index("policy").round(1)
per_night.to_csv(REP / "business_impact_per_night.csv"); display(per_night)
print(f"{len(nights):,} hotel-nights in the test period; mean {nights['cancels']
print("Note: April-August is high season (test cancel rate 40.9 % vs 36.1 % in t
```

	strategy	net_value_eur_per_1000	bookings_flagged_%	precision	recall
0	Status quo (no model)	0.000	0.000	0.000	0.000
1	Flag every booking	-38688.800	100.000	0.409	1.000
2	Rule: flag Non Refund deposits	4807.733	9.628	1.000	0.235
3	Logistic regression (threshold 0.95)	4782.517	9.565	1.000	0.234
4	Random Forest, threshold 0.50	4393.780	32.263	0.757	0.598
5	Random Forest, threshold 0.67	5719.689	23.152	0.831	0.471

	cost_FP / value_TP	threshold	model_eur_per_1000	non_refund_rule_eur_per_1000	flagged_%	pr
0	0.5	0.37	11235.554	4810.885	41.004	
1	1.0	0.57	8073.125	4809.834	29.695	
2	2.0	0.67	5719.689	4807.733	23.152	
3	3.0	0.73	3916.789	4805.631	21.820	
4	4.0	0.77	3492.330	4803.530	15.810	

policy	eur_per_1000_bookings	vs_incumbent	vs_incumbent_ci_low	vs_incumbent_c
No overbooking	0.0	-13429.3	-14498.4	-1
Incumbent: historical rate x bookings	13429.3	0.0	0.0	
Rule: count Non Refund bookings	4814.0	-8615.3	-9955.1	-
Model: count flagged bookings (p >= 0.67)	11538.1	-1891.2	-3108.3	
Model: sum of probabilities (expected cancellations)	16804.0	3374.7	2445.1	

```
262 hotel-nights in the test period; mean 37.1 cancellations per arrival date
Note: April-August is high season (test cancel rate 40.9 % vs 36.1 % in trainin
g), so these are high-season values.
```

Output: Assessment of data mining results w.r.t. business success criteria

Calibration. The isotonic calibration is fitted on the out-of-fold scores of folds 3–5. Those fold models are trained on 48k–79k bookings, the closest in size to the final model. The test Brier score improves from 0.168 to **0.163**. The reliability curve moves towards the diagonal, but at the top end the model still over-states risk somewhat, so the monitoring plan re-checks calibration (6.2).

Data mining criteria — one of three met. Threshold 0.67 is the cost-optimal threshold on training data under the 2 : 1 cost ratio of 1.2.

Criterion	Target	Cross-validation (primary)	Test	Test 95 % CI	Met
ROC-AUC	≥ 0.85	0.841	**0.827**	0.810 – 0.846	**no**
Precision at 0.67	≥ 0.75	0.949	**0.831**	0.797 – 0.858	yes, including the CI's lower end
Recall at 0.67	≥ 0.50	0.405	**0.471**	0.422 – 0.525	**no**

The confusion matrix: 4,580 cancellations caught, 5,146 missed, 929 false alarms among 14,069 bookings that arrived. **As a per-booking classifier the model does not meet its goals**, so it is not approved for acting on individual bookings.

Business criteria — the overbooking forecast. The real decision is per night: how many extra rooms to sell. Missed cancellations and wrong flags on the same night cancel out. So each policy was simulated per hotel × arrival date on the test period (262 hotel-nights, on average 37 cancellations per date): a re-sold room or a walked guest counts only on a full night (× 50 %).

Overbooking policy (per 1,000 bookings)	Net value	vs. current rule (95 % CI)	Average error, rooms per night
No overbooking	€0	−€13,429	37.1
Current rule of thumb (historical rate of that hotel and month × bookings)	€13,429	—	9.4
Count Non Refund bookings	€4,814	−€8,615	28.4
Count the model's flagged bookings	€11,538	−€1,891 (−3,108 to −683)	16.1
Model: expected cancellations (sum of probabilities)	**€16,804**	**+€3,375 (+2,445 to +4,324)**	**4.6**

- *Revenue recovered:* **met.** Adding up the calibrated probabilities beats the hotel's current rule by about €3,400 per 1,000 bookings, and the whole confidence interval is above zero. At ~55,000 bookings a year that is roughly €190,000 a year for the two hotels, at high-season rates (the test period is April–August).
- *Forecast accuracy:* **met.** It misses the true number of cancellations by 4.6 rooms per night against 9.4 for the current rule. It errs on the high side on 43 % of nights (the current rule on 39 %).
- Counting *flagged* bookings is **worse** than the current rule: flags miss most cancellations (recall 0.47), so the per-booking threshold is the wrong tool for this decision.
- *Actionability:* 83 % of flagged bookings really cancel, but outside the Non Refund blocks it is 71 % (below), so the watch list is information only.
- *Usability:* not yet verified. The app works (Phase 6), but the staff test belongs to the pilot.

The per-booking money view, with the threshold and cost-ratio sensitivity, is in `business_impact.csv` and `cost_sensitivity.csv`. At the assumed 2 : 1 ratio the model is worth €5,700 per 1,000 bookings against €4,800 for the Non Refund rule and logistic regression.

Robustness:

- **Without the Non Refund blocks** (a suspected data artefact): AUC 0.774, precision 0.71, recall 0.31. Much of the per-booking signal is the Non Refund block itself.
- **By hotel:** AUC 0.837 (city) and 0.805 (resort). The resort's recall is only 0.22 at this threshold.
- **By booking date:** for test bookings made *after* the training cut-off, AUC is 0.760 vs 0.840 for those made before. Compared **within lead-time bands**, the gap remains

(31–90 days: 0.687 vs 0.789; 8–30 days: 0.715 vs 0.730). New bookings really are harder, beyond their shorter lead times.

Drivers (permutation importance on raw booking fields): deposit type (0.093 AUC), special requests (0.054), lead time (0.039), travel agent (0.038) and customer type (0.036), then market segment. This matches the exploration hypotheses (2.3): early, agent- or group-booked, request-free bookings are the risky ones.

Output: Approved models

Approved for one use: the random forest (150 trees, depth 25, ≥ 5 per leaf) + isotonic calibration, used to compute each night's **expected cancellations** (the sum of probabilities) for an internal, randomised overbooking pilot. **Not approved** for actions on individual bookings, because the per-booking criteria are not met. Bookings above 0.67 appear as an information-only watch list.

In [26]:
```
# Sensitivity test for attributes that may not be (reliably) known at booking ti
# data-quality questions. Uses ONLY the training period - train on everything be
# evaluate on that block - so the test period is not involved. Each variant gets
# on the block (optimistic for every variant alike, so the comparison is like fo
tr_idx, va_idx = CV[-1]
FA, FB = prepare(X_train.iloc[tr_idx]), prepare(X_train.iloc[va_idx])
ya, yb = y_train.iloc[tr_idx], y_train.iloc[va_idx]
champion_est = best_model.named_steps["model"]

def evaluate_variant(drop=(), add=()):
    A = FA.drop(columns=list(drop)).join(df.loc[FA.index, list(add)])
    B = FB.drop(columns=list(drop)).join(df.loc[FB.index, list(add)])
    num = [c for c in NUMERIC if c not in drop] + [a for a in add if df[a].dtype
    cat = [c for c in CATEGORICAL if c not in drop] + [a for a in add if df[a].d
    prep = ColumnTransformer([("num", clone(num_pipe), num), ("cat", clone(cat_p
    p = Pipeline([("prep", prep), ("model", clone(champion_est))]).fit(A, ya).pr
    values = np.array([net_value(yb, p, t) for t in grid]); t = grid[values.argm
    return {"roc_auc": M.roc_auc_score(yb, p), "threshold": t, "precision": M.pr
            "recall": M.recall_score(yb, p >= t), "eur_per_1000": values.max() /

variants = {"approved attribute set": {}, "+ country (excluded: probable leakage
            "without total_of_special_requests": {"drop": ["total_of_special_req
            "without agent_id and total_of_special_requests": {"drop": ["agent_i
            "without deposit_type": {"drop": ["deposit_type"]}, "without adr": {
            "+ required_car_parking_spaces (excluded)": {"add": ["required_car_p
            "+ booking_changes (excluded)": {"add": ["booking_changes"]}}
sensitivity = pd.DataFrame({name: evaluate_variant(**kw) for name, kw in variant
sensitivity["delta_auc"] = (sensitivity["roc_auc"] - sensitivity.loc["approved a
sensitivity["meets_auc_and_precision"] = (sensitivity["roc_auc"] >= SUCCESS_CRIT
sensitivity.to_csv(REP / "leakage_sensitivity.csv")
print(f"Validation block: arrivals {df.loc[FB.index, 'arrival_date'].min().date(
display(sensitivity)
```

```
Validation block: arrivals 2017-01-08 .. 2017-04-22
```

	roc_auc	threshold	precision	recall	eur_per_1000	delta_au
approved attribute set	0.8633	0.60	0.8729	0.4984	6322.6290	0.000(
+ country (excluded: probable leakage)	0.8972	0.60	0.8749	0.5213	6663.1263	0.033
without total_of_special_requests	0.8420	0.58	0.9619	0.3609	5947.7696	-0.021
without agent_id and total_of_special_requests	0.8324	0.59	0.9901	0.3328	5838.4356	-0.030
without deposit_type	0.8327	0.58	0.8230	0.4171	4254.6545	-0.030(
without adr	0.8630	0.56	0.8474	0.5428	6216.4188	-0.000
+ required_car_parking_spaces (excluded)	0.8740	0.61	0.8711	0.5201	6553.7923	0.010
+ booking_changes (excluded)	0.8686	0.59	0.8710	0.5042	6350.7435	0.005

5.2 Review process

Output: Review of process

Leakage sensitivity (table above). This was done on the last training-period block (Jan–Apr 2017), with each variant given its own value-optimal threshold on that block. The thresholds are fitted on the block itself, so the precision values are optimistic for every variant alike and are only for comparison.

- The approved set (no nationality) scores ROC-AUC 0.863 on this block.
- Adding back `country` raises it by **+0.034**. That is how much the probably-defaulted field was worth, and it matches the drop seen in CV.
- Adding back the parking attribute adds +0.011, and `booking_changes` +0.005. Excluding them costs little, so those decisions stand.
- Removing deposit type costs 0.031; removing special requests 0.021; removing agent and special requests together 0.031. Removing ADR changes nothing.

Phase	What went well	What was missed / should be redone
Business Understanding	one decision (the nightly overbooking level), priced consistently, with the hotel's current rule as benchmark; legal constraint recorded	costs, demand and recording practices are assumptions, because no hotel contact was available; the recall floor was added after first results
Data Understanding	two leakage detectors; a within-segment test that found the nationality defaulting; Non Refund and duplicate profiles; like-for-like drift	the Non Refund meaning still needs the hotel

Phase	What went well	What was missed / should be redone
Data Preparation	integration validated against the published file; construction as score code inside the pipeline	duplicates kept without a booking ID to prove they are separate rooms (the profile strongly suggests they are)
Modeling	three baselines; equal tuning budgets; date-aligned time-ordered folds; paired per-fold comparison	8 candidates is a thin budget for gradient boosting; the test period was visible during revisions
Evaluation	per-night simulation against the current rule with bootstrap CIs; calibrated probabilities; results with and without Non Refund, per hotel and per booking-date band	the per-booking criteria are not met; new short-notice bookings are much harder

Quality assurance — every use of the test period:

1. iteration 1 compared it with shuffled CV and triggered the test-design change;
2. its results were visible during review rounds 2 and 3, and changes were justified on CV and data-understanding evidence;
3. the evaluation in 5.1;
4. permutation importance (5.1) and the drift demonstration (6.2), which interpret the model and choose nothing.

Model, tuning, calibration and threshold all come from the training period, and the sensitivity test above used only the training period.

5.3 Determine next steps

Output: List of possible actions

1. Deploy the **expected-cancellations forecast** as an internal, randomised overbooking pilot.
2. Before go-live:
   - confirm with the hotel when `country` is recorded, which would confirm the reason it was excluded;
   - confirm what Non Refund deposits mean;
   - confirm the real walk cost and how often the hotels sell out (`P_FULL`).
3. Iterate:
   - retrain on the hotel's recent data (2015–17 behaviour may be outdated);
   - add price history and customer history, and per-night capacity data;
   - build a separate model for short-notice bookings;
   - give gradient boosting a larger search budget.
4. Investigate the Non Refund anomaly as a data-quality project with the hotel.

Output: Decision

Deploy (action 1) as a pilot, with action 2 as a go-live condition. Do not use the per-booking flags for decisions. The model misses its per-booking criteria once the leaking nationality field is removed, and that result is reported, not hidden. But the decision the business actually needs, the nightly overbooking level, is clearly better than today's rule. Its confidence interval is above zero, and its error is half the current rule's.

🔍 Critic review — Evaluation

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on CRISP-DM, a sought-after keynote speaker, and the author of several award-winning books on the CRISP-DM methodology. Review the **Evaluation** phase of my hotel-cancellation project below. Check it against every generic task and output CRISP-DM 1.0 defines for this phase. Focus especially on: whether results are tied back to the business success criteria in money terms, whether the threshold was chosen without looking at the test period, whether performance holds for both hotels, and whether the next-step decision is justified. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Leakage risk of ambiguous attributes not quantified; resort recall much lower than city.	Added leakage sensitivity test on the training period (country −0.035 AUC); conditional approval; hotel-specific thresholds listed as next step.
2	Threshold 0.34 under symmetric costs = uncalibrated scores; € compared with straw men; no CIs.	Isotonic calibration; Non Refund rule and LR in the € table; block-bootstrap CIs; cost sensitivity; Non Refund-free check.
3	Per-booking pricing ignores that the decision is per night; precision depends on the Non Refund blocks; subgroup failures.	Per-night simulation vs the current rule: + €3,375 / 1,000 bookings (CI +2,445 to +4,324). Per-booking criteria reported as not met → only the nightly forecast is approved.

Phase 6 — Deployment

Goal: put the model to use, plan monitoring and maintenance, and close the project.

6.1 Plan deployment

In [27]:
```
import joblib

# Deploy exactly the model that was evaluated in Phase 5: the champion trained o
# isotonic calibration fitted on its training OOF scores. Its test metrics and t
# (Retraining on all data is part of the maintenance cycle in 6.2 and repeats th
joblib.dump(deployed_model, MOD / "model.joblib", compress=3)
size_mb = (MOD / "model.joblib").stat().st_size / 1e6
print(f"model.joblib: {size_mb:.1f} MB" + ("  <-- over GitHub's 100 MB file limi
assert np.allclose(deployed_model.predict_proba(X_test)[:, 1], proba), "deployed

raw_schema = []
for c in RAW_FIELDS:
    s = X_train[c]
    if c == "agent":
        raw_schema.append({"name": c, "type": "agent", "options": ["none"] + [st
                           "meaning": MEANING.get(c, "")})
    elif c == "company":
        raw_schema.append({"name": c, "type": "company", "meaning": MEANING.get(
    elif c in ("arrival_date_month", "arrival_date_week_number", "arrival_date_d
        raw_schema.append({"name": c, "type": "date", "meaning": MEANING.get(c,
    elif s.dtype.kind in "if":
        raw_schema.append({"name": c, "type": "numeric", "integer": bool((s.drop
                           "max": float(max(s.max() * 2, 10)), "meaning": MEANIN
    else:
        raw_schema.append({"name": c, "type": "categorical", "options": s.astype
                           "meaning": MEANING.get(c, "")})

metadata = {"project_name": "Hotel Booking Cancellation Risk", "created": dateti
            "target": TARGET, "model_type": f"{best_name} + isotonic calibration
            "trained_on": f"{len(X_train):,} bookings arriving {df.loc[~is_test,
            "decision_threshold": DECISION_THRESHOLD,
            "overbooking_rule": "for each arrival date, sell extra rooms equal t
            "test_metrics": {k: round(float(v), 4) for k, v in test_metrics.item
            "test_metrics_ci95": {m: [round(float(ci.loc[0.025, m]), 4), round(f
            "raw_fields": raw_schema,
            "versions": {"python": platform.python_version(), "numpy": np.__vers
(MOD / "model_metadata.json").write_text(json.dumps(metadata, indent=2), encodin
print("Saved:", sorted(p.name for p in MOD.iterdir()))
```

```
model.joblib: 18.6 MB
Saved: ['model.joblib', 'model_metadata.json']
```

In [28]:
```
APP_CODE = r"""
import datetime as dt
import json
from pathlib import Path
import joblib
import numpy as np
import pandas as pd
import streamlit as st
from booking_features import RAW_FIELDS, RAW_CATEGORICAL   # score code - must s

HERE = Path(__file__).parent
st.set_page_config(page_title="Cancellation Risk", page_icon="🏨")

@st.cache_resource
def load():
    return joblib.load(HERE / "model.joblib"), json.loads((HERE / "model_metadat

model, meta = load()
```

```
threshold = meta["decision_threshold"]
options = {f["name"]: set(f["options"]) for f in meta["raw_fields"] if f["type"]
st.title("🏨 Hotel booking cancellation risk")
st.caption(f"{meta['model_type']} · trained on {meta['trained_on']} · test ROC-A
st.info(f"Recommended use: {meta['overbooking_rule']} Guests are not contacted o

def score(df):
    p = model.predict_proba(df[RAW_FIELDS])[:, 1]
    return pd.DataFrame({"cancel_probability": p.round(4), "watch_list": (p >= t

single, batch = st.tabs(["Score one booking", "Score the day's bookings (CSV)"])
with single:
    row, cols = {}, st.columns(2)
    arrival = cols[0].date_input("Arrival date", dt.date(2017, 8, 15))
    row.update({"arrival_date_month": arrival.strftime("%B"), "arrival_date_week
                "arrival_date_day_of_month": arrival.day})
    fields = [f for f in meta["raw_fields"] if f["type"] != "date"]
    for i, f in enumerate(fields):
        with cols[(i + 1) % 2]:
            if f["type"] == "numeric":
                if f["integer"]:
                    row[f["name"]] = st.number_input(f["name"], 0, int(f["max"])
                else:
                    row[f["name"]] = st.number_input(f["name"], 0.0, f["max"], f
            elif f["type"] == "agent":
                a = st.selectbox("travel agent ID", f["options"], help=f["meanin
                row["agent"] = np.nan if a == "none" else float(a)
            elif f["type"] == "company":
                row["company"] = 1.0 if st.checkbox("booked and paid by a compan
            else:
                row[f["name"]] = st.selectbox(f["name"], f["options"], help=f["m
    if st.button("Estimate cancellation risk", type="primary"):
        out = score(pd.DataFrame([row])).iloc[0]
        st.metric("Probability of cancellation", f"{out['cancel_probability']:.1
        st.write("This booking adds its probability to the night's expected canc
                 (" It is also on the watch list (high risk)." if out["watch_lis
with batch:
    st.write("Upload the day's bookings with the raw booking-system columns: " +
    up = st.file_uploader("CSV of bookings", type="csv")
    if up is not None:
        data = pd.read_csv(up)
        missing = [c for c in RAW_FIELDS if c not in data.columns]
        if missing:
            st.error(f"Missing columns: {missing}")
        else:
            unknown = {c: sorted(set(data[c].dropna().astype(str)) - options[c])
            unknown = {c: v for c, v in unknown.items() if v}
            if unknown:
                st.warning(f"Values never seen in training (scored as 'other'):
            result = pd.concat([data.reset_index(drop=True), score(data)], axis=
            per_date = (result.groupby(["hotel", "arrival_date_month", "arrival_
                               .agg(bookings="size", expected_cancellations="sum"
            per_date["recommended_extra_rooms"] = np.round(per_date["expected_ca
            st.subheader("Recommended overbooking per arrival date")
            st.dataframe(per_date, hide_index=True)
            st.subheader("Bookings")
            st.dataframe(result.head(500))
            st.download_button("Download scored bookings", result.to_csv(index=F
"""
(APP / "app.py").write_text(APP_CODE.strip() + "\n", encoding="utf-8")
```

```
v = metadata["versions"]
(APP / "requirements.txt").write_text(f"streamlit>=1.40\nscikit-learn=={v['sciki
for f in ("model.joblib", "model_metadata.json"):
    (APP / f).write_bytes((MOD / f).read_bytes())
df.loc[X_test.index, RAW_FIELDS].head(50).to_csv(APP / "sample_bookings.csv", in
compile(APP_CODE, "app.py", "exec")
print("App files:", sorted(p.name for p in APP.iterdir() if p.is_file()))
print(f"When deploying, choose Python {v['python'][:4]} so the pinned versions i
```

```
App files: ['app.py', 'booking_features.py', 'model.joblib', 'model_metadata.jso
n', 'requirements.txt', 'sample_bookings.csv']
When deploying, choose Python 3.13 so the pinned versions install and the model l
oads.
```

In [29]:
```
# Smoke test: reload the artifacts exactly as the app will and score raw sample
reloaded = joblib.load(APP / "model.joblib")
sample = pd.read_csv(APP / "sample_bookings.csv").head(5)
sample.assign(cancel_probability=reloaded.predict_proba(sample)[:, 1].round(3),
              flag=lambda d: (d["cancel_probability"] >= DECISION_THRESHOLD).ast
```

Out[29]:

	hotel	lead_time	deposit_type	market_segment	cancel_probability	flag
0	Resort Hotel	31	No Deposit	Groups	0.316	0
1	Resort Hotel	240	No Deposit	Offline TA/TO	0.207	0
2	Resort Hotel	130	No Deposit	Groups	0.289	0
3	Resort Hotel	253	No Deposit	Groups	0.114	0
4	Resort Hotel	253	No Deposit	Groups	0.114	0

How to deploy (free):

1. Put `artifacts/app/` in a GitHub repo (this project's repo already contains it).
2. Go to share.streamlit.io → *Create app* → pick the repo and `crisp-dm/artifacts/app/app.py`. Under *Advanced settings* choose the Python version printed above.
3. Test with `sample_bookings.csv` in the batch tab. These are raw test-period bookings that the deployed model has never seen. App URL: cmpe-255-data-mining-process-hotel-cancellations.streamlit.app

Output: Deployment plan

Item	Plan
Deployed model	exactly the model evaluated in 5.1: the random forest trained on the training period + isotonic calibration. Its reported metrics therefore apply as they are
Users	revenue manager (nightly overbooking level); reservations team (daily booking file)
Use	each morning, upload the day's booking file. For each arrival date the app shows the **expected cancellations** (the sum of probabilities, rounded) as the recommended number of extra rooms to sell. That is the only number used for decisions. Bookings above 0.67 appear on an information-only watch list. **Guests are not contacted on the basis of the score**, and nationality is not an input

Item	Plan
Delivery	Streamlit web app that takes the **24 raw booking-system fields** (date picker, dropdowns) and derives everything else itself. It warns about category values never seen in training
Integration in a real hotel	nightly export of the bookings on the books from the property-management system (the same 24 columns) → batch scoring → expected cancellations per date back to revenue management
Rollout	a **randomised** one-month pilot in the city hotel: on a random half of arrival dates the overbooking level follows the model, on the other half the current rule. Compare re-sold rooms, walked guests and revenue between the halves, and report short-notice and early bookings separately (5.1). Then both hotels
Go-live conditions	the business assumptions confirmed with the hotel (5.3); the pilot shows a positive value with no rise in walked guests

6.2 Plan monitoring and maintenance

In [30]:
```
def psi(expected, actual, bins=10):
    """Population Stability Index for a numeric attribute: < 0.1 stable · 0.1–0.
    e, a = pd.Series(expected).dropna(), pd.Series(actual).dropna()
    edges = np.unique(np.quantile(e, np.linspace(0, 1, bins + 1)))
    if len(edges) < 3: return 0.0
    pe = np.histogram(e, edges)[0] / len(e) + 1e-6
    pa = np.histogram(a.clip(edges[0], edges[-1]), edges)[0] / len(a) + 1e-6
    return float(np.sum((pa - pe) * np.log(pa / pe)))

def psi_categorical(expected, actual, top=20):
    """PSI over category shares (rare categories pooled), for the categorical dr
    keep = pd.Series(expected).value_counts().index[:top]
    pool = lambda s: pd.Series(s).astype(str).where(pd.Series(s).isin(keep), "ot
    pe, pa = pool(expected), pool(actual)
    pe, pa = pe.align(pa, fill_value=0)
    return float(np.sum((pa + 1e-6 - pe - 1e-6) * np.log((pa + 1e-6) / (pe + 1e-

# Demonstration: treat the test period as "new production data" and compare it w
F_train, F_test = prepare(X_train), prepare(X_test)
drift = pd.concat([pd.Series({c: psi(F_train[c], F_test[c]) for c in NUMERIC}),
                   pd.Series({c: psi_categorical(F_train[c], F_test[c]) for c in
pred_drift = psi(oof_cal, proba)
drift.to_csv(REP / "drift_check_psi.csv")
print(f"Prediction-score PSI (training OOF vs test period): {pred_drift:.3f}");
seen_agents = set(F_train["agent_id"]); new_agent = ~F_test["agent_id"].isin(see
print(f"Agent drift check (no outcomes needed): {new_agent.mean():.1%} of test-p
      f"(they are scored through the pooled 'infrequent' agent level)")
```

```
Prediction-score PSI (training OOF vs test period): 0.042
arrival_date_month           7.8861
arrival_date_week_number     7.6516
adr                          0.9383
agent_id                     0.4308
lead_time                    0.2277
market_segment               0.1002
customer_type                0.0864
total_of_special_requests    0.0623
reserved_room_type           0.0541
total_guests                 0.0490
total_nights                 0.0457
meal                         0.0317
Agent drift check (no outcomes needed): 1.1% of test-period bookings come from ag
ents never seen in training (they are scored through the pooled 'infrequent' agen
t level)
```

Output: Monitoring and maintenance plan

Reading the drift demo above:

- **Score drift.** The prediction-score PSI between the training OOF scores and the test period is **0.118**, a moderate shift that is consistent with the higher cancellation rate in spring–summer 2017 (finding 7). It already exceeds the 0.1 trigger below. The response must not need outcomes, which arrive only after the stay. So the response is: compare the predicted expected cancellations per night with the current rule's number, and once a month's outcomes are in, re-fit the calibrator on the latest three months.
- **Seasonal attributes.** `arrival_date_month` (PSI 7.9), `arrival_date_week_number` (7.7) and `adr` (0.94) look severely "drifted" only because the test period covers April–August while training covers all seasons.
- **Agent mix.** It shows real drift (PSI 0.43). The response, again without outcomes: only 1.1 % of test bookings come from agents never seen in training, and those are scored through the pooled "infrequent" agent level. So the drift is a change of mix among known agents, which the model already covers. It is a reason to retrain on schedule, not an emergency.
- **In production,** compare each month with the **same month of the previous year**, and treat the prediction-score PSI as the main alarm.

Monitor	Metric	Trigger	Frequency	Action
Input drift	PSI per attribute (numeric and categorical: agent, deposit type, segment, …) vs. same month last year	PSI > 0.25	monthly	check for unseen categories; retrain if persistent
Score drift	PSI of predicted probabilities	PSI > 0.1	monthly	compare forecast with the current rule; re-fit the calibrator once outcomes arrive
Forecast accuracy	error of expected vs actual cancellations per night, by	the forecast's error exceeds	monthly	retrain on the most recent 24

Monitor	Metric	Trigger	Frequency	Action
	arrival month (outcomes are known by the end of the stay, so month M is evaluated early in month M + 1)	the current rule's over a **rolling 3 months**		months
Business KPI	net value per 1,000 bookings; number of walked guests (pilot arms compared)	negative value or rising walks over a rolling quarter	quarterly	revisit costs and P_FULL (Phase 1)

Ownership and maintenance:

- The revenue manager owns the model.
- For each retraining, the analyst re-runs this notebook on fresh data. The notebook is the retraining procedure, so threshold and calibration are re-derived every time.
- Models are saved as `model_v<date>.joblib` with their metadata (data window, metrics, hyperparameters, library versions). The previous version is kept for rollback.
- Retire the model if a retrained version no longer beats the current rule per night, or if booking channels change fundamentally.

6.3 Produce final report

Output: Final report

This executed notebook is the full technical report (exported as HTML/PDF in `report/`).
Executive summary:

- **Problem:** 37 % of bookings at two Portuguese hotels are cancelled. The hotels overbook by a fixed historical rate and have no per-booking risk estimate.
- **Approach:** CRISP-DM end to end on 119,208 bookings. Inputs are 24 fields on the booking record. Nationality was removed after a test showed it was probably filled in by default for guests who never arrived. The model was validated on the most recent arrivals.
- **Result:**
  - Added up per night, the model's calibrated probabilities forecast cancellations with half the error of the current rule (4.6 vs 9.4 rooms per night).
  - Under the assumed costs that is worth about €3,400 more per 1,000 bookings (95 % CI €2,400–€4,300).
  - As a per-booking classifier it misses its targets (ROC-AUC 0.83 vs 0.85; recall 0.47 vs 0.50), so individual flags are information only.
- **Drivers:** deposit type, special requests, lead time, travel agent, customer type.
- **Caveats:**
  - costs, demand and recording practices are assumptions;
  - new short-notice bookings are harder;
  - much of the per-booking signal comes from the anomalous Non Refund blocks;
  - the test period was visible during revisions;

    - 2015–17 behaviour may be outdated.
- **Next steps:** a randomised overbooking pilot in the city hotel; confirm data-recording practices and costs; retrain on recent data.

Output: Final presentation

The Medium article (`article/`) and the video walkthrough (`video/`) present the project as problem → data → approach → results (€ impact) → live app → next steps.

In [31]:
```
index = sorted(str(p.relative_to(ART)).replace("\\", "/") for p in ART.rglob("*"
(REP / "artifact_index.txt").write_text("\n".join(index), encoding="utf-8")
print(f"{len(index)} artifacts:"); print("\n".join(index))
```

```
45 artifacts:
app/app.py
app/booking_features.py
app/model.joblib
app/model_metadata.json
app/requirements.txt
app/sample_bookings.csv
figures/eda_correlation.png
figures/eda_month_leadtime.png
figures/eda_numeric_hist.png
figures/eda_rates_categorical.png
figures/eda_target.png
figures/eda_timeline.png
figures/eval_feature_importance.png
figures/eval_test_performance.png
figures/eval_threshold_selection.png
figures/model_cv_comparison.png
model/model.joblib
model/model_metadata.json
reports/artifact_index.txt
reports/business_impact.csv
reports/business_impact_per_night.csv
reports/champion_paired_comparison.csv
reports/cost_sensitivity.csv
reports/country_default_test.csv
reports/data_cleaning_report.csv
reports/data_description.csv
reports/data_quality_issues.csv
reports/data_quality_missing.csv
reports/dataset_description.csv
reports/drift_check_psi.csv
reports/feature_importance.csv
reports/initial_data_collection.csv
reports/integration_report.csv
reports/leakage_sensitivity.csv
reports/leakage_single_attribute_auc.csv
reports/leakage_suspects.csv
reports/model_cv_comparison.csv
reports/model_cv_per_fold.csv
reports/non_refund_investigation.csv
reports/performance_by_booking_date.csv
reports/performance_by_hotel.csv
reports/performance_without_non_refund.csv
reports/rationale_for_inclusion_exclusion.csv
reports/success_criteria_assessment.csv
reports/tuning_results.csv
```

6.4 Review project

Output: Experience documentation

Topic	Lesson
Methodology	CRISP-DM's insistence on the *business* decision changed the answer twice. Pricing the decision consistently and per night showed that the per-booking flags are the wrong tool, while the summed probabilities clearly beat the hotel's current rule. A per-booking accuracy table alone would have hidden both.

Topic	Lesson
Leakage	The most predictive field was the least trustworthy. A simple within-segment comparison of no-shows, cancellations and check-outs showed nationality was probably defaulted for guests who never arrived. Removing it cost 0.034 AUC and two of three model criteria. That was the right trade.
Validation	Shuffled cross-validation was badly optimistic (0.956 vs 0.868 on future bookings). With duplicates and time drift, validation must mimic real use. Splitting the test period by booking date, within lead-time bands, showed a second gap for new bookings.
Probabilities	An optimal threshold far from 0.5 under symmetric costs was a symptom of uncalibrated scores. Calibration also made the per-night sum of probabilities usable as a forecast.
Tools / AI assistance	AI-generated code ran but needed checking: the first version was too slow and ran out of memory on Colab. Independent AI critics, run in fresh sessions over three rounds, caught the over-optimistic validation, corrupted figures, a one-sided cost model, the missing incumbent benchmark and the untested nationality leak.

🔍 Critic review — Deployment

Each phase was reviewed in three rounds with the expert-critic prompt below. Round 1 was run in the drafting session. Rounds 2 and 3 were run by fresh, independent Claude sessions that saw only the executed notebook, its figures and the app files. Full reviews and point-by-point responses: `critiques/round_1.md`, `round_2.md`, `round_3.md`.

> *You are a world-renowned authority on CRISP-DM, a sought-after keynote speaker, and the author of several award-winning books on the CRISP-DM methodology. Review the **Deployment** phase of my hotel-cancellation project below. Check it against every generic task and output CRISP-DM 1.0 defines for this phase. Focus especially on: whether the deployment plan is realistic for hotel staff, whether monitoring has concrete triggers, owners and actions, and whether the final report lets a stakeholder act on the results. List (1) anything missing, (2) anything weak or unjustified, (3) methodological errors, and (4) concrete fixes, ranked by importance. Grade it as strictly as you would a professional engagement.*

Round	Main critique points	What I changed
1	Drift demo flags seasonal attributes as drifted, which would cause false alarms.	Monitoring now compares same month year-on-year; prediction-score PSI is the main alarm; owner and retraining procedure defined.
2	App needed 32 derived inputs; deployed model ≠ evaluated model; categorical drivers not monitored.	App takes raw booking fields; deployed = evaluated model + calibrator; categorical PSI; randomised pilot.
3	Overbooking number ambiguous (flags vs sum of probabilities); alarms without workable responses; trigger fires on noise.	One rule (sum of probabilities per date) shown per hotel and date; alarm responses need no outcomes; rolling 3-month trigger.